

Riadenie a umelá inteligencia

Obsah

1	Prípadová štúdia 1 – Piecka	4
1.1	Znenie	4
1.2	Analytické riešenie	5
1.3	Programový kód (Matlab/Simulink)	6
1.3.1	a) Otvorená slučka – Simulácia výstupu $y(t)$	6
1.3.2	b) Návrh PI regulátorov pomocou MATLAB funkcií	7
1.3.3	c) Simulácia uzavretej riadiacej štruktúry – výstup $y(t)$ a akcna velicina $u(t)$	11
1.3.4	d) Diskretizácia prenosovej funkcie $G_p(s)$ pomocou c2d	13
1.4	Výstupy zo simulácie	14
1.4.1	Otvorená slučka – výstup $y(t)$	14
1.4.2	Uzavretá slučka – výstupy $y(t)$ pre rozne metódy	14
1.4.3	Akcna velicina $u(t)$ pre rozne metódy	15
2	Prípadová štúdia 2 – Piecka	18
2.1	Prípadová štúdia 2 – Piecka a motor	18
2.2	Zadanie – Prípadová štúdia 2: Motor	18
2.3	Analytické riešenie	19
2.4	Programový kód (Matlab/Simulink)	20
2.4.1	a) Model systému a návrh PI regulátorov	20
2.4.2	a) Simulácia spojitého systému s PI regulátormi	20
2.4.3	b1) Michajlovovo kritérium a Ziegler–Nichols	21
2.4.4	b2) Export výstupov pre dokumentáciu	21
2.4.5	b3) Diskretizácia systému a výpočet PSD koeficientov	21
2.4.6	b4) Výpočet prenosov $Y/W(z)$ a $U/W(z)$	21
2.4.7	b5) Výpočet $y(\infty)$, $u(\infty)$ a diferenčných rovníc	21
2.4.8	b6) Simulácia URO s PSD regulátorom	22
2.4.9	b7) Porovnanie spojitého a diskrétného výstupu	22
2.5	Použité MATLAB funkcie	23
2.6	Výstupy zo simulácie	27
2.6.1	Výstup $y(t)$ – porovnanie: piecka vs motor (spojitý PI regulátor) .	27
2.6.2	Výstup $y(k)$ – porovnanie: piecka vs motor (diskrétny PSD regulátor)	28
2.7	Modely regulačných štruktúr	29
2.7.1	Simulink model – PI regulátor (spojitý čas)	29
2.7.2	Simulink model – PSD regulátor (diskrétny čas)	30
3	Prípadová štúdia 3 – Piecka, Motor, Výmenník	31
3.1	Znenie	31
3.2	Analytické riešenie	32
3.3	Hlavné skripty pre jednotlivé systémy	33
3.4	Použité MATLAB funkcie	39
3.5	Výstupy zo simulácie	42
3.6	Modely regulačných štruktúr v Simulinku	48

4	Prípadová štúdia 4 – Jednosmerný motor	50
4.1	Znenie	50
4.2	Analytické riešenie	51
4.3	Programový kód (Matlab/Simulink)	52
4.4	Výstupy zo simulácie	54
4.4.1	Odozva výstupnej veličiny $y(t)$	54
4.4.2	Riadiaca veličina $u(t)$	55
4.4.3	Regulačná odchýlka $e(t)$	55
4.4.4	Simulink model – PP regulátor	56

Listings

1	Simulácia systému bez regulátora (model msim.slx)	6
2	Výpočty PI parametrov pomocou rôznych metód	7
3	Funkcia NaslinPI pre výpočet PI regulátora	8
4	Funkcia MomPI pre výpočet PI regulátora	8
5	Funkcia ZiegNicPI pre výpočet regulátora	9
6	Funkcia GhPI pre výpočet PI regulátora (Graham–Lathrop)	9
7	Funkcia ButtwPI pre výpočet PI regulátora (Butterworth)	10
8	Simulacia vystupu $y(t)$ pre rozne PI regulatore	11
9	Porovnanie akčnej velicity $u(t)$ pre rozne PI regulatore	12
10	Diskretizacia prenosovej funkcie pomocou c2d	13
11	Model motora a návrh PI regulátorov	20
12	Simulácia PI regulátorov v spojitom čase	20
13	Výpočet podľa Michajlovovho kritéria a ZN	21
14	Export grafov pre dokumentáciu	21
15	Diskretizácia a PSD výpočet	21
16	Prenosové funkcie URO	21
17	Ustálené hodnoty a diferenčné rovnice	22
18	Simulácia PSD systému	22
19	Porovnanie $y(t)$ vs $y(kT)$	22
20	Definícia diskretizovaného modelu motora	39
21	Definícia systému pece 2. rádu	39
22	Diskretizácia systému výmenníka 3. rádu	39
23	Výpočet dead-beat regulátora bez saturácie	40
24	Dead-beat regulátor s obmedzením na $u(0)$	40
25	Simulácia systému so zadaným regulátorom	40
26	Výpočet ustálených hodnôt	41

1 Prípadová štúdia 1 – Piecka

1.1 Znenie

Cieľom tejto úlohy je analyzovať a navrhnuť riadiaci systém pre elektrickú pec, ktorej dynamika je modelovaná prenosovou funkciou druhého rádu. Úloha vychádza z cvičení 3 a 4 a je zameraná na návrh spojitého PI (resp. PID) regulátora pomocou rôznych metód nepriamej syntézy, ako aj na simuláciu systému v otvorenej a uzavretej regulačnej slučke.

Zadanie pozostáva z nasledujúcich bodov:

- **a)** Vykresliť priebeh regulovanej veličiny $y(t)$ v otvorenej slučke (model v Simulinku). Použiť prenos podľa cvičenia 3 (slidy 2–3).
- **b)** Určiť koeficienty PID regulátora (alternatívne P, PI, PD) s využitím metód nepriamej syntézy:
 - **b1)** Naslinova metóda (výpočet + funkcia v Matlabe)
 - **b2)** Metóda optimálneho modulu (MOM) (výpočet + funkcia v Matlabe)
 - **b3)** Metóda Ziegler–Nichols (modifikácia Z–N1 alebo Z–N2, výpočet + funkcia v Matlabe)
 - **b4)** Metóda štandardných tvarov (Graham–Lathrop a Butterworthove mnohočleny)
 - výpočet + funkcia v Matlabe
- **c)** V uzavretej riadiacej štruktúre URO s PI/PID regulátorom navrhnutým podľa jednotlivých metód overiť stabilitu a kvalitu v ustálenom stave pomocou simulácie:
 - **c1)** Vykresliť priebeh $y(t)$ a $u(t)$ pri skokovo sa meniacej referenčnej veličine $w(t)$ (napr. 0.8, 1, 1.2) alebo pri konštantnom $w(t)$ s pôsobením poruchy. Výsledné priebehy porovnať podľa grafov uvedených na slidoch cvičenia.
- **d)** Prepočítať prenosovú funkciu procesu $G_P(s)$ na diskretný tvar $G_P(z)$ pre dve rôzne hodnoty vzorkovacej periódy (overenie cez funkciu c2d).

1.2 Analytické riešenie

1.3 Programový kód (Matlab/Simulink)

1.3.1 a) Otvorená slučka – Simulácia výstupu $y(t)$

Skript vykonáva simuláciu systému v otvorenej slučke, bez spätnej väzby a bez regulátora. Systém je modelovaný prenosovou funkciou $G_p(s) = \frac{0.1}{s^2+s+0.2}$. Pomocou Simulinku (msim.slx) sa zisťuje časový priebeh výstupu $y(t)$ pri jednotkovom riadiacom vstupe $w(t)$. Výsledok ukazuje dynamiku neregulovaného systému, vrátane oneskorenia, prekmitu a ustálenia.

```
1  tspan = [0 70];
2  step_t = 5;
3  u_des = 1;
4  desr = [35 40];
5  desr_val = 0.5;
6
7  % Definicia spojiteho modelu systemu
8  num = 0.1;
9  den = [1 1 0.2];
10 tffun = tf(num, den);
11
12 % Simulacia v Simulinku (model msim.slx)
13 simdata = sim("msim.slx");
14 y = simdata.simout.Data(:, 1);
15 u = simdata.simout.Data(:, 2);
16 t = simdata.tout;
17
18 % Vystup systemu v otvorenej slucke
19 figure("Name", "System feedback on desired w(t)");
20 plot(t, y, 'g', 'LineWidth', 2);
21 hold on;
22 plot(t, u, 'r', 'LineWidth', 2);
23 grid on;
24 ylim([0 max(max(y, u)) + 0.5]);
25 legend("System output y(t)", "Desired signal w(t)");
26 hold off;
```

Listing 1: Simulácia systému bez regulátora (model msim.slx)

1.3.2 b) Návrh PI regulátorov pomocou MATLAB funkcií

Tento skript vypočítava parametre PI regulátorov pre daný systém pomocou piatich odlišných metód nepriamej syntézy. Každá metóda vychádza z iného princípu:

- **Naslinova metóda** využíva požadované preregulovanie systému na výpočet parametrov.
- **Metóda optimálneho modulu (MOM)** navrhuje regulátor tak, aby uzavretý systém mal dobré tlmenie a dynamickú odozvu.
- **Ziegler–Nichols** je experimentálna metóda založená na určení kritického zosilnenia a periódy oscilácie.
- **Graham–Lathrop** a **Butterworth** metódy využívajú štandardizované polynómy na modelovanie požadovaného dynamického správania.

Výsledkom je päť sád PI parametrov, ktoré sú ďalej použité v simulačnom modeli regulovaného systému.

```
1 %% === b) NAVRH KOEFICIENTOV PI REGULATORA ROZNYMI METODAMI ===
2 % Vsetky vypocty a volania MATLAB funkcii podľa požiadaviek
3
4 % b1) Metoda Naslin
5 PINaslinCoef = NaslinPI(tffun, 1.75);
6
7 % b2) Metoda optimalneho modulu (MOM)
8 MomPICoef = MomPI(tffun);
9
10 % b3) Metoda Ziegler-Nichols (1. modifikacia)
11 ZiegNicPICoef = ZiegNicPI(tffun, 'PI');
12
13 % b4) Metoda standardnych tvarov
14 GhPICoef = GhPI(num, den);          % Graham Lathrop
15 ButtwPICoef = ButtwPI(num, den);% Butterworth
```

Listing 2: Výpočty PI parametrov pomocou rôznych metód

Jednotlivé funkcie použité v skripte sú uvedené nižšie.

b1) Naslinova metóda počíta parametre PI regulátora pomocou algebraických rovníc vyplývajúcich z požiadaviek na preregulovanie (max. prekmit). Využíva symbolické spracovanie rovníc pre získanie koeficientov regulátora.

```

1 function Reg = NaslinPI(tfun, alpha)
2     syms s r0 r_m1;
3     [num, den] = tfdata(tfun);
4
5     G_p = poly2sym(cell2mat(num), s) / poly2sym(cell2mat(den), s)
6         ;
7     G_r = r0 + r_m1 / s;
8     eq = 1 + (G_p * G_r) == 0;
9     eq_simplified = expand(lhs(eq) * poly2sym(cell2mat(den), s));
10    eq_final = collect(expand(eq_simplified * s), s);
11    coeffs_list = coeffs(eq_final, s, 'All');
12    eq1 = coeffs_list(2)^2 == alpha * coeffs_list(1) *
13        coeffs_list(3);
14    eq2 = coeffs_list(3)^2 == alpha * coeffs_list(2) *
15        coeffs_list(4);
16    sol = solve([eq1, eq2], [r0, r_m1]);
17
18    Reg.P = double(sol.r0);
19    Reg.I = double(sol.r_m1);
20 end

```

Listing 3: Funkcia NaslinPI pre výpočet PI regulátora

b2) Metóda optimálneho modulu (MOM) navrhuje PI regulátor na základe normalizovanej prenosovej funkcie systému. Cieľom je získať dobre tlmenú a rýchlu odozvu v uzavretej slučke. Výpočty využívajú vzťahy medzi koeficientmi menovateľa normalizovaného systému.

```

1 function Reg = MomPI(tfun)
2     sys = tfnorm(tfun);
3     a1 = sys.den(2);
4     a2 = sys.den(3);
5     a3 = 0;
6
7     r0 = (1/(2*sys.num)) * ((a1^3 - 2*a1*a2 + a3)/(a1*a2 - a3));
8     r_m1 = (1/(2*sys.num)) * ((a1^2 - a2)/(a1*a2 - a3));
9
10    Reg.P = r0;
11    Reg.I = r_m1;
12 end

```

Listing 4: Funkcia MomPI pre výpočet PI regulátora

b3) Metóda Ziegler–Nichols (1. modifikácia) vychádza z odozvy systému na skokový vstup. Určia parametre oneskorenia a nárastu (T_u , T_n), pre nastavenie P , I .

```

1 function Reg = ZiegNicPI(tfun, type)
2     K_p = tfnorm(tfun).num;
3     T = tfresp(tfun);
4
5     switch type
6         case "P"
7             Reg.P = T.n / (K_p*T.u);
8         case "PI"
9             Reg.P = 0.9 * (T.n / (K_p*T.u));
10            Reg.I = 3.5 * T.u;
11         case "PD"
12            Reg.P = 1.2 * (T.n / (K_p*T.u));
13            Reg.D = 0.25 * T.u;
14         case "PID"
15            Reg.P = 1.25 * (T.n / (K_p*T.u));
16            Reg.I = 2 * T.u;
17     end
18 end

```

Listing 5: Funkcia ZiegNicPI pre výpočet regulátora

b4) Metóda štandardných tvarov – Graham–Lathrop navrhuje regulátor porovnaním charakteristického polynómu systému. Výsledné parametre PI regulátora sa získavajú riešením symbolických rovníc podľa požadovanej dynamiky systému.

```

1 function Reg = GhPI(num, den)
2     syms s r0 r_m1 w0
3
4     G_p = poly2sym(num, s) / poly2sym(den, s);
5     G_r = r0 + r_m1 / s;
6     eq = 1 + G_p * G_r == 0;
7     eq_simplified = expand(lhs(eq) * poly2sym(den, s));
8     eq_final = collect(expand(eq_simplified * s), s);
9     coeffs_list = coeffs(eq_final, s, 'All');
10
11     c3 = coeffs_list(1);
12     c2 = coeffs_list(2);
13     c1 = coeffs_list(3);
14     c0 = coeffs_list(4);
15
16     w0 = solve(c2 == 1.75 * w0, w0, 'Real', true);
17     w0 = double(w0);
18     Kp = solve(c1 == 2.15 * w0^2, r0, 'Real', true);
19     Reg.P = double(Kp);
20     Ti = solve(c0 == w0^3, r_m1, 'Real', true);
21     Reg.I = double(Ti);
22 end

```

Listing 6: Funkcia GhPI pre výpočet PI regulátora (Graham–Lathrop)

b4) Metóda štandardných tvarov – Butterworth navrhuje tvar uzavretej prenosovej funkcie s cieľom dosiahnuť maximálne plochú frekvenčnú charakteristiku bez prekmitu v pásme priepustnosti. Výpočet PI regulátora sa realizuje porovnaním charakteristického polynómu s Butterworthovým predpisom.

```

1 function Reg = ButtwPI(num, den)
2     syms s r0 r_m1 w0
3     G_p = poly2sym(num, s) / poly2sym(den, s);
4     G_r = r0 + r_m1 / s;
5     eq = 1 + G_p * G_r == 0;
6
7     eq_simplified = expand(lhs(eq) * poly2sym(den, s));
8     eq_final = collect(expand(eq_simplified * s), s);
9
10    coeffs_list = coeffs(eq_final, s, 'All');
11    mult = double(1 / coeffs_list(1));
12    coeffs_list = coeffs_list * mult;
13
14    c2 = coeffs_list(2);
15    c1 = coeffs_list(3);
16    c0 = coeffs_list(4);
17
18    w0 = solve(c2 == 2 * w0, w0, 'Real', true);
19    w0 = double(w0);
20
21    Kp = solve(c1 == 2 * w0^2, r0, 'Real', true);
22    Reg.P = double(Kp);
23
24    Ti = solve(c0 == w0^3, r_m1, 'Real', true);
25    Reg.I = double(Ti);
26 end

```

Listing 7: Funkcia ButtwPI pre výpočet PI regulátora (Butterworth)

1.3.3 c) Simulacia uzavretej riadiacej struktury – vystup $y(t)$ a akcna velicina $u(t)$

Nasledujuci skript vykonava simuláciu riadeného systému so zapojeným PI regulátorom. Pre každú z piatich metód návrhu regulátora sa sleduje časový priebeh systémovej výstupnej veličiny $y(t)$ a zodpovedajúcej akcie $u(t)$. Referenčný signál $w(t)$ obsahuje skokové zmeny (napr. 0.8, 1.0, 1.2) bez poruchy. Porovnaním priebehov môžeme hodnotiť stabilitu, prekmitý a čas ustavenia každého prístupu.

```
1 % Simulacia PI regulatorov      vystup y(t)
2 figure("Name", "PI Regulators      Output y(t)");
3 stairs(t, u, 'k--', 'LineWidth', 2); % w(t)      cierna
   prerusovana
4 hold on;
5 grid on;
6 title("PI Regulators comparison      System Output y(t)");
7
8 % Naslin
9 Reg = PINaslinCoef;
10 data = sim("reg.slx");
11 nas_t = data.tout;
12 nas_y = data.simdata.Data;
13 nas_u = data.acout.Data;
14 plot(nas_t, nas_y, 'r', 'LineWidth', 2);
15
16 % MOM
17 Reg = MomPICoef;
18 data = sim("reg.slx");
19 mom_t = data.tout;
20 mom_y = data.simdata.Data;
21 mom_u = data.acout.Data;
22 plot(mom_t, mom_y, 'g', 'LineWidth', 2);
23
24 % Ziegler Nichols
25 Reg = ZiegNicPICoef;
26 data = sim("reg.slx");
27 zn_t = data.tout;
28 zn_y = data.simdata.Data;
29 zn_u = data.acout.Data;
30 plot(zn_t, zn_y, 'b', 'LineWidth', 2);
31
32 % Graham Lathrop
33 Reg = GhPICoef;
34 data = sim("reg.slx");
35 gl_t = data.tout;
36 gl_y = data.simdata.Data;
37 gl_u = data.acout.Data;
38 plot(gl_t, gl_y, 'm', 'LineWidth', 2);
39
40 % Butterworth
41 Reg = ButtwPICoef;
```

```

42 data = sim("reg.slx");
43 bw_t = data.tout;
44 bw_y = data.simdata.Data;
45 bw_u = data.acout.Data;
46 plot(bw_t, bw_y, 'c', 'LineWidth', 2);
47
48 legend("w(t)", "Naslin", "MOM", "Ziegler-Nichols", "Graham-
    Lathrop", "Butterworth", "Location", "southeast");
49 hold off;

```

Listing 8: Simulacia vystupu $y(t)$ pre rozne PI regulatore

Simulacia potvrdzuje, ze rozne metody vedu k rozdielnemu dynamickemu spravaniu. Naslin a MOM poskytnu rychle ustalenie s miernym prekmitom, zatiaľ čo Ziegler–Nichols vedie k agresívnejšiemu nábehu. Butterworthova metóda je najviac tlmená, ale s najpomalsim priebehom. Graham–Lathrop poskytuje kompromis medzi rýchlosťou a stabilitou.

```

1 % Akčná veľkosť u(t)
2 figure("Name", "Action signal u(t)");
3 plot(nas_t, nas_u, 'r', 'LineWidth', 2);
4 hold on;
5 plot(mom_t, mom_u, 'g', 'LineWidth', 2);
6 plot(zn_t, zn_u, 'b', 'LineWidth', 2);
7 plot(gl_t, gl_u, 'm', 'LineWidth', 2);
8 plot(bw_t, bw_u, 'c', 'LineWidth', 2);
9 grid on;
10 legend("Naslin", "MOM", "Ziegler-Nichols", "Graham-Lathrop", "
    Butterworth", "Location", "southeast");
11 hold off;

```

Listing 9: Porovnanie akčnej veľkosti $u(t)$ pre rozne PI regulatore

1.3.4 d) Diskretizacia prenosovej funkcie $G_p(s)$ pomocou c2d

Cielom tejto časti je prevod spojitej prenosovej funkcie $G_p(s)$ do diskrétného tvaru $G_p(z)$ pre dve rôzne hodnoty periódy vzorkovania. Použita je Tustinova aproximácia (bilinéarna transformácia), ktorá zachováva stabilitu a frekvencnú odozvu v určitom rozsahu.

Použitý model:

$$G_p(s) = \frac{0.1}{s^2 + s + 0.2}$$

Pre vzorkovacie periódy $T = 0.5$ s a $T = 0.15$ s získame nasledujúce diskkrétne modely:

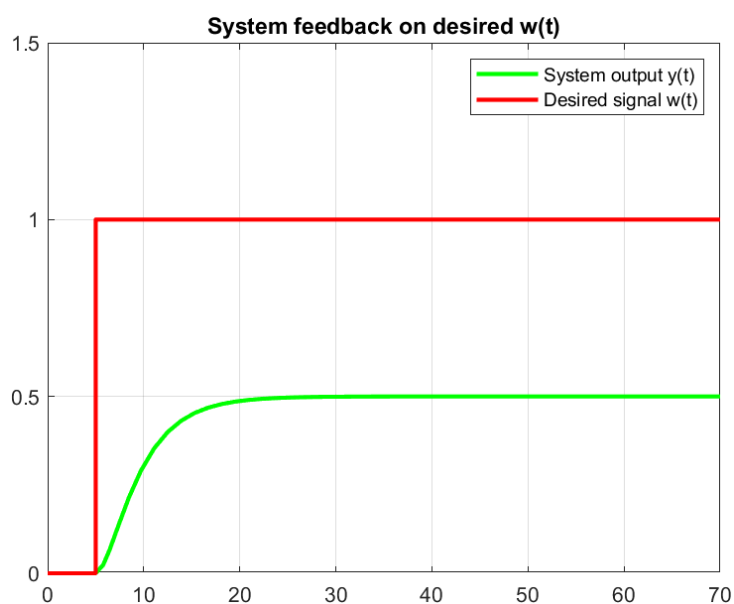
```
1 % Prenosova funkcia procesu
2 num = 0.1;
3 den = [1 1 0.2];
4 tffun = tf(num, den);
5
6 % Diskretizacia Tustinova metoda
7 dtfun1 = c2d(tffun, 0.5, 'tustin');
8 disp("Diskretizovany system (T = 0.5s):");
9 disp(dtfun1);
10
11 dtfun2 = c2d(tffun, 0.15, 'tustin');
12 disp("Diskretizovany system (T = 0.15s):");
13 disp(dtfun2);
```

Listing 10: Diskretizacia prenosovej funkcie pomocou c2d

Vystupom sú dve prenosové funkcie vo forme $G_p(z)$, ktoré môžu byť následne použité v digitalnej implementácii regulátora alebo simulácii diskkrétneho modelu v Simulinku.

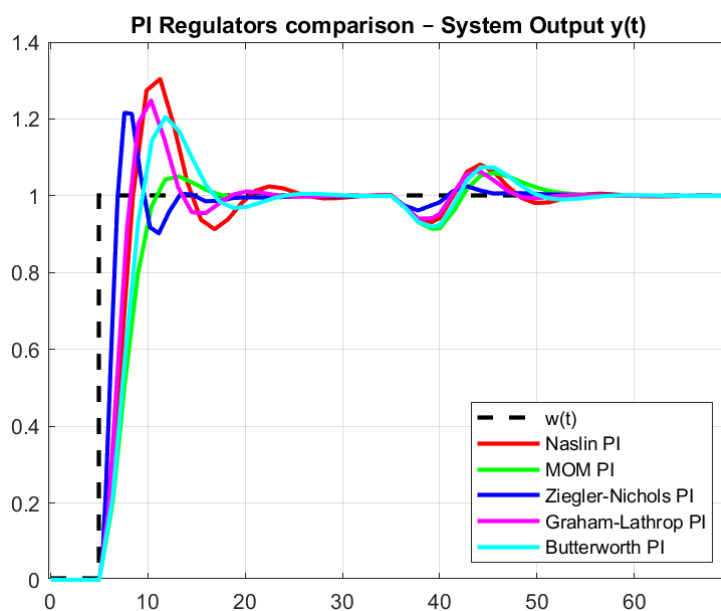
1.4 Vystupy zo simulacie

1.4.1 Otvorena slučka – vystup $y(t)$



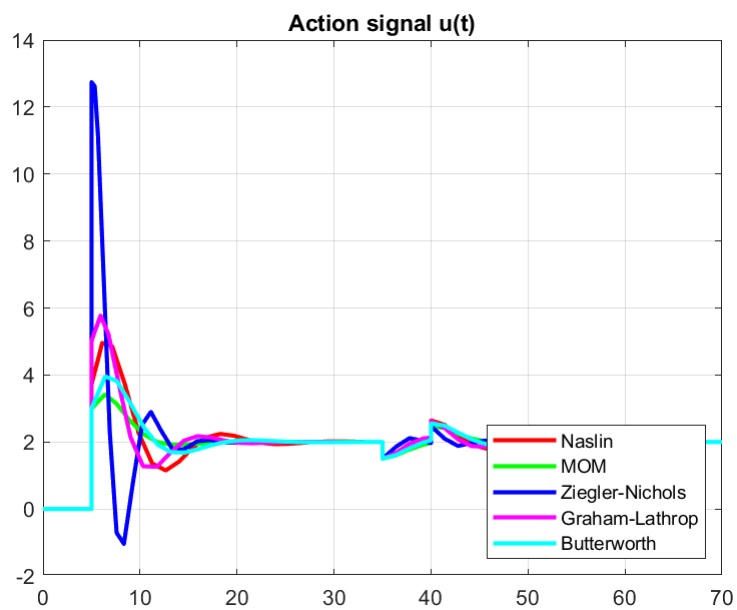
Obr. 1: Vystup $y(t)$ systému bez regulácie v otvorenej slučke

1.4.2 Uzavretá slučka – vystupy $y(t)$ pre rôzne metódy

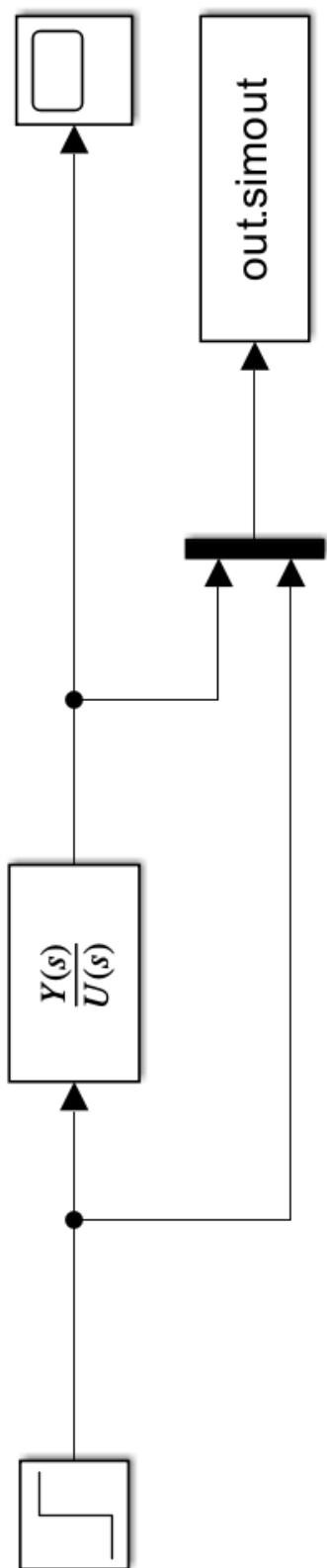


Obr. 2: Porovnanie vystupu $y(t)$ pre PI regulátore (Naslin, MOM, Ziegler-Nichols, G-L, Butterworth)

1.4.3 Akcna velicina $u(t)$ pre rozne metody



Obr. 3: Akcna velicina $u(t)$ pre PI regulatore – porovnanie metod



Obr. 4: Simulink model – otevřená slučka (msim.slx)

2 Prípadová štúdia 2 – Piecka

2.1 Prípadová štúdia 2 – Piecka a motor

Úlohou je riešiť riadenie uhlovej rýchlosti jednosmerného motora podľa zadania z cvičení 5 a 6. Na základe predchádzajúcej štúdie (PS1) treba:

2.2 Zadanie – Prípadová štúdia 2: Motor

a) Aplikovať všetky postupy z Prípadovej štúdie 1 (PS1) spolu s rozšíreniami na riešenie komplexnej úlohy pre **motor** (t. j. riadenie uhlovej rýchlosti jednosmerného motora) z cvičenia 6. Doplniť výpočty, podľa potreby aktualizovať a zovšeobecniť funkcie a simulačné schémy v MATLABe/Simulinku.

b) Rozšíriť postupy z PS1 o:

- b1) Určenie kritického zosilnenia / periódy Michajlovovým alebo Nyquistovým kritériom v príslušnej modifikácii metódy Ziegler–Nichols (výpočet + aktualizácia funkcie v MATLABe).
- b2) Použiť vhodný spôsob exportu grafov zo simulačných schém za účelom vytvárania dokumentácie (skript / funkcia v MATLABe).
- b3) Prepočet PID / PSD pre oba systémy (piecka a motor): nájsť koeficienty q_0, q_1, q_2 PSD (P / PS / PD) regulátora na základe už získaných koeficientov PID regulátora pomocou lichobežníkovej aproximácie pri vhodnej voľbe T_{vz} (výpočet + funkcie v MATLABe) – viď cvičenie 5, slide 3 (piecka) a cvičenie 6, slide 13 (motor).
- b4) Pomocou už získaného diskrétného tvaru prenosovej funkcie $F_p(z)$ získať prenosovú funkciu URO $Y/W(z)$ a prenosovú funkciu riadiaceho zásahu $U/W(z)$ vzhľadom na regulačný obvod s PSD regulátorom.
- b5) Výpočet ustálených hodnôt: riadenej veličiny $y(\infty)$, akčnej veličiny $u(\infty)$ a diferenciálnu rovnicu URO z oboch prenosov (výpočet + funkcie v MATLABe) – viď cvičenie 3, slidy 15–16; cvičenie 4, slidy 4–7 (piecka); cvičenie 6, slidy 14–16 (motor).
- b6) V regulačnej štruktúre URO s PSD regulátorom simuláciou overiť stabilitu a kvalitu v ustálenom stave, t. j. vykresliť priebeh $y(k)$, $u(k)$ a $e(k)$ pre skokovo sa meniacu riadiacu veličinu $w(t)$ s nulovou poruchou, resp. konštantnú riadiacu veličinu s pôsobením poruchy, prípadne iné kombinácie.
- b7) Porovnať $y(t)/y(kT)$ a $u(t)/u(kT)$ v URO pre vhodne zvolenú T_{vz} (simulačné schéma v Simulinku).

2.3 Analytické riešenie

2.4 Programový kód (Matlab/Simulink)

2.4.1 a) Model systému a návrh PI regulátorov

V tejto časti bol modelovaný jednosmerný motor so spojitou prenosovou funkciou $G_p(s)$ v tvare:

$$G_p(s) = \frac{K_1}{(T_1 s + 1)s}$$

kde $K_1 = 100$ a $T_1 = 0.15$. Na základe tohto modelu boli navrhnuté tri PI regulátory rôznymi metódami (Naslin, Butterworth, Graham–Lathrop).

```
1 K1 = 100;
2 T1 = 0.15;
3 num = K1;
4 den = conv([T1 1], [1 0]);
5 tfun = tf(num, den);
6
7 NaslinReg = NaslinPI(tfun, 2.2);
8 ButtwReg = ButtwPI(num, den);
9 GhReg = GhPI(num, den);
```

Listing 11: Model motora a návrh PI regulátorov

2.4.2 a) Simulácia spojitého systému s PI regulátormi

Simulácia bola vykonaná v Simulinku so všetkými tromi PI regulátormi. Výstupy sú porovnané v jednom grafe.

```
1 Reg = NaslinReg;
2 data = sim("reg.slx");
3 nas_t = data.tout;
4 simout = data.simout.Data(:, 1);
5 data_naslin = data.simout.Data(:, 2);
6
7 stairs(nas_t, simout, '--', 'LineWidth', 2); hold on;
8 plot(nas_t, data_naslin, "LineWidth", 2);
9
10 Reg = ButtwReg;
11 data = sim("reg.slx");
12 data_buttw = data.simout.Data(:, 2);
13 plot(data.tout, data_buttw, "LineWidth", 2);
14
15 Reg = GhReg;
16 data = sim("reg.slx");
17 data_gh = data.simout.Data(:, 2);
18 plot(data.tout, data_gh, "LineWidth", 2);
19
20 legend("w(t)", "Naslin PI", "Butterworth PI", "Graham-Lathrop PI");
```

Listing 12: Simulácia PI regulátorov v spojitom čase

2.4.3 b1) Michajlovovo kritérium a Ziegler–Nichols

Použitím funkcie `MicCr` sa určilo kritické zosilnenie a perióda pre motor, následne ZN odhad PI regulátora.

```
1 [r0_kr, omega_kr, T_kr] = MicCr(tfun);  
2 Kp_zn = 0.45 * r0_kr;  
3 Ti_zn = T_kr / 1.2;
```

Listing 13: Výpočet podľa Michajlovovho kritéria a ZN

2.4.4 b2) Export výstupov pre dokumentáciu

Grafy boli uložené do PNG pomocou `saveas()` na účely dokumentácie v LaTeXu.

```
1 saveas(gcf, 'fig_PI_Regulator.png');  
2 saveas(gcf, 'fig_Gyw.png');  
3 saveas(gcf, 'fig_Guw.png');  
4 saveas(gcf, 'fig_PSD_Regulator.png');  
5 saveas(gcf, 'fig_Porovnanie_y.png');
```

Listing 14: Export grafov pre dokumentáciu

2.4.5 b3) Diskretizácia systému a výpočet PSD koeficientov

Spojité model $G_p(s)$ bol diskretizovaný Tustinovou metódou s $T_{vz} = 0.4$ s. Následne sa vypočítali koeficienty PSD regulátora.

```
1 T_vz = 0.4;  
2 dtf = c2d(tfun, T_vz, 'tustin');  
3 PSDStruct = PSD(NaslinReg, T_vz);  
4 q0 = PSDStruct.q0;  
5 q1 = PSDStruct.q1;  
6 q2 = PSDStruct.q2;
```

Listing 15: Diskretizácia a PSD výpočet

2.4.6 b4) Výpočet prenosov $Y/W(z)$ a $U/W(z)$

Na základe diskretizovaných modelov sa určili prenosové funkcie výstupu a akcie:

```
1 Gp_z = dtf;  
2 Gr_z = tf([q0 q1], [1 -1], T_vz);  
3 Gyw = feedback(Gp_z * Gr_z, 1);  
4 Guw = feedback(Gr_z, 1 + Gp_z * Gr_z);
```

Listing 16: Prenosové funkcie URO

2.4.7 b5) Výpočet $y(\infty)$, $u(\infty)$ a diferenčných rovníc

Z ustálených hodnôt a koeficientov boli vypočítané diferenčné rovnice URO.

```

1 y_inf = dcgain(Gyw);
2 u_inf = dcgain(Guw);
3
4 [num_y, den_y] = tfdata(Gyw, 'v');
5 [num_u, den_u] = tfdata(Guw, 'v');

```

Listing 17: Ustálené hodnoty a diferenčné rovnice

2.4.8 b6) Simulácia URO s PSD regulátorom

V tejto časti bola overená funkčnosť regulátora v diskkrétnej forme.

```

1 data = sim("dreg.slx");
2 nas_t = data.tout;
3 simout = data.simout.Data(:, 1);
4 data = data.simout.Data(:, 2);
5 stairs(nas_t, simout, '--'); hold on;
6 stairs(nas_t, data);

```

Listing 18: Simulácia PSD systému

2.4.9 b7) Porovnanie spojitého a diskrétného výstupu

Pre overenie správnosti diskretizácie sa porovnali výstupy $y(t)$ a $y(kT)$.

```

1 data_cont = sim("reg.slx");
2 y_cont = data_cont.simout.Data(:,2);
3 t_cont = data_cont.tout;
4
5 data_disc = sim("dreg.slx");
6 y_disc = data_disc.simout.Data(:,2);
7 t_disc = data_disc.tout;
8
9 plot(t_cont, y_cont);
10 stairs(t_disc, y_disc);

```

Listing 19: Porovnanie $y(t)$ vs $y(kT)$

2.5 Použité MATLAB funkcie

NaslinPI.m – Výpočet PI regulátora pomocou Naslinovej metódy

Táto funkcia navrhuje PI regulátor na základe Naslinovej metódy, využívajúc algebraické vzťahy medzi koeficientmi charakteristickej rovnice pre dané požadované tlmenie (parametrom alpha).

```
1 function Reg = NaslinPI(tfun, alpha)
2     syms s r0 r_m1;
3     [num, den] = tfdata(tfun);
4
5     G_p = poly2sym(cell2mat(num), s) / poly2sym(cell2mat(den), s)
6         ;
7     G_r = r0 + r_m1 / s;
8     eq = 1 + (G_p * G_r) == 0;
9
10    eq_simplified = expand(lhs(eq) * poly2sym(cell2mat(den), s));
11    eq_final = collect(expand(eq_simplified * s), s);
12    coeffs_list = coeffs(eq_final, s, 'All');
13
14    eq1 = coeffs_list(2)^2 == alpha * coeffs_list(1) *
15          coeffs_list(3);
16    eq2 = coeffs_list(3)^2 == alpha * coeffs_list(2) *
17          coeffs_list(4);
18
19    sol = solve([eq1, eq2], [r0, r_m1]);
20
21    Reg.P = double(sol.r0);
22    Reg.I = double(sol.r_m1);
23 end
```

ButtwPI.m – Výpočet PI regulátora podľa Butterworthovej aproximácie

Používa Butterworthov štandardný polynóm pre návrh PI regulátora zabezpečujúceho dobre tlmený systém bez prekmitov v pásme priepustnosti.

```
1 function Reg = ButtwPI(num, den)
2     syms s r0 r_m1 w0
3     G_p = poly2sym(num, s) / poly2sym(den, s);
4     G_r = r0 + r_m1 / s;
5     eq = 1 + G_p * G_r == 0;
6
7     eq_simplified = expand(lhs(eq) * poly2sym(den, s));
8     eq_final = collect(expand(eq_simplified * s), s);
9
10    coeffs_list = coeffs(eq_final, s, 'All');
11    mult = double(1 / coeffs_list(1));
12    coeffs_list = coeffs_list * mult;
13
14    c2 = coeffs_list(2);
15    c1 = coeffs_list(3);
```



```

16     c0 = coeffs_list(4);
17
18     w0 = solve(c2 == 2 * w0, w0, 'Real', true);
19     w0 = double(w0);
20
21     Kp = solve(c1 == 2 * w0^2, r0, 'Real', true);
22     Reg.P = double(Kp);
23
24     Ti = solve(c0 == w0^3, r_m1, 'Real', true);
25     Reg.I = double(Ti);
26 end

```

GhPI.m – Graham–Lathrop metóda pre PI regulátor

Používa Graham–Lathrop štandardný polynóm. Cieľom je dosiahnuť požadovanú dynamiku pomocou vzťahov medzi koeficientmi.

```

1 function Reg = GhPI(num, den)
2     syms s r0 r_m1 w0
3     G_p = poly2sym(num, s) / poly2sym(den, s);
4     G_r = r0 + r_m1 / s;
5     eq = 1 + G_p * G_r == 0;
6
7     eq_simplified = expand(lhs(eq) * poly2sym(den, s));
8     eq_final = collect(expand(eq_simplified * s), s);
9     coeffs_list = coeffs(eq_final, s, 'All');
10    mult = double(1 / coeffs_list(1));
11    coeffs_list = coeffs_list * mult;
12
13    c2 = coeffs_list(2);
14    c1 = coeffs_list(3);
15    c0 = coeffs_list(4);
16
17    w0 = solve(c2 == 1.75 * w0, w0, 'Real', true);
18    w0 = double(w0);
19
20    Kp = solve(c1 == 2.15 * w0^2, r0, 'Real', true);
21    Reg.P = double(Kp);
22
23    Ti = solve(c0 == w0^3, r_m1, 'Real', true);
24    Reg.I = double(Ti);
25 end

```

MicCr.m – Kritické zosilnenie podľa Michajlovovho kritéria

Numericky vypočíta kritické zosilnenie a periódu oscilácie podľa fázovej odozvy systému.

```

1 function [r0_kr, omega_kr, T_kr] = MicCr(Gp)
2     [num, den] = tfdata(Gp, 'v');
3     omega = linspace(0.01, 100, 10000);
4

```

```

5      s = 1j * omega;
6      Gp_eval = polyval(num, s) ./ polyval(den, s);
7
8      phase = angle(Gp_eval);
9      [~, idx] = min(abs(phase + pi));
10
11     omega_kr = omega(idx);
12     mag = abs(Gp_eval(idx));
13     r0_kr = 1 / mag;
14     T_kr = 2 * pi / omega_kr;
15 end

```

PSD.m – Výpočet PSD koeficientov z PI regulátora

Prevod spojitého PI regulátora do diskkrétnej PSD formy pomocou lichobežníkovej (Tustinovej) aproximácie.

```

1 function ZReg = PSD(SReg, Ts)
2     ZReg.q0 = SReg.P * (1 + (Ts / (2 * SReg.I)));
3     ZReg.q1 = -SReg.P * (1 - (Ts / (2 * SReg.I)));
4     ZReg.q2 = 0;
5 end

```

ssPI.m – Výpočet $y(\infty)$

Funkcia vypočíta ustálenú hodnotu výstupu $y(t)$ pre jednotkový vstup a PI regulátor.

```

1 function y_inf = ssPI(tf, Reg)
2     syms s r0 r_m1
3
4     G_r = r0 + r_m1 / s;
5     [num, den] = tfdata(tf);
6     G_p = poly2sym(cell2mat(num), s) / poly2sym(cell2mat(den), s)
7         ;
8
9     G_r = subs(G_r, [r0 r_m1], [Reg.P Reg.I]);
10
11     eq = (G_p * G_r) / (1 + (G_p * G_r));
12     y_inf = double(limit(s * eq * 1/s, s, 0));
13 end

```

ciPI.m – Výpočet $u(\infty)$

Analogicky k ssPI, funkcia určuje ustálenú hodnotu akčnej veličiny $u(t)$.

```

1 function u_inf = ciPI(tf, Reg)
2     syms s r0 r_m1
3
4     G_r = r0 + r_m1 / s;
5     [num, den] = tfdata(tf);

```

```

6      G_p = poly2sym(cell2mat(num), s) / poly2sym(cell2mat(den), s)
7      ;
8      G_r = subs(G_r, [r0 r_m1], [Reg.P Reg.I]);
9
10     eq = G_r / (1 + (G_p * G_r));
11     u_inf = double(limit(s * eq * 1/s, s, 0));
12 end

```

dtftust.m – Symbolická Tustinova diskretizácia $G(s) \rightarrow G(z)$

Prevod spojitej prenosovej funkcie do diskrétnej domény pomocou Tustinovej substitúcie.

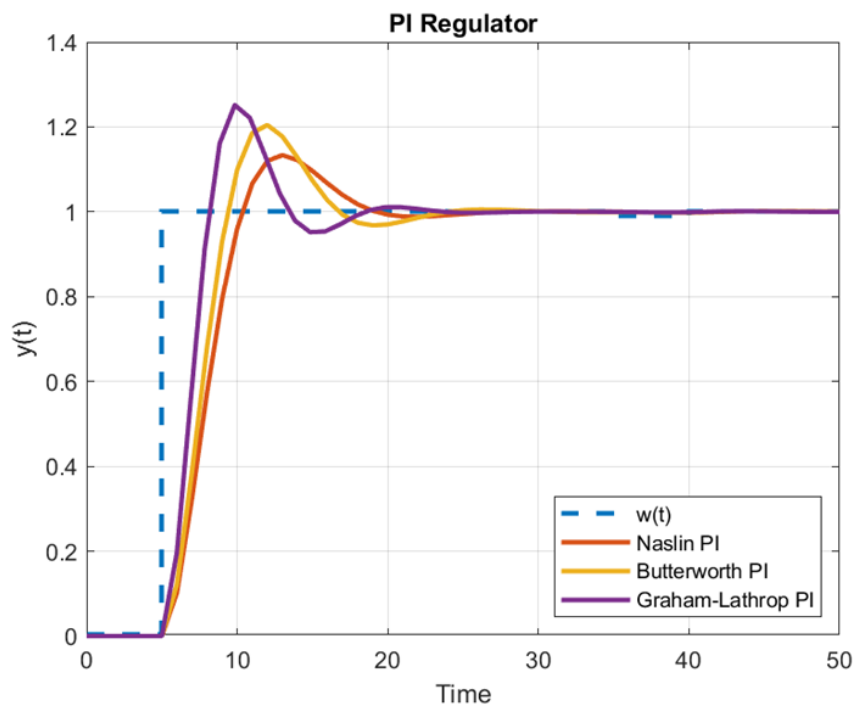
```

1 function out = dtftust(tf, Ts)
2     syms s z;
3
4     snum = poly2sym(cell2mat(tf.Numerator), s);
5     sden = poly2sym(cell2mat(tf.Denominator), s);
6     s_to_z = (2 / Ts) * ((z - 1) / (z + 1));
7
8     [num, den] = numden(subs(snum / sden, s, s_to_z));
9     num_coeffs = flip(double((coeffs(num, z))));
10    den_coeffs = flip(double((coeffs(den, z))));
11
12    num_coeffs = num_coeffs / den_coeffs(1);
13    den_coeffs = den_coeffs / den_coeffs(1);
14
15    out.num = num_coeffs;
16    out.den = den_coeffs;
17 end

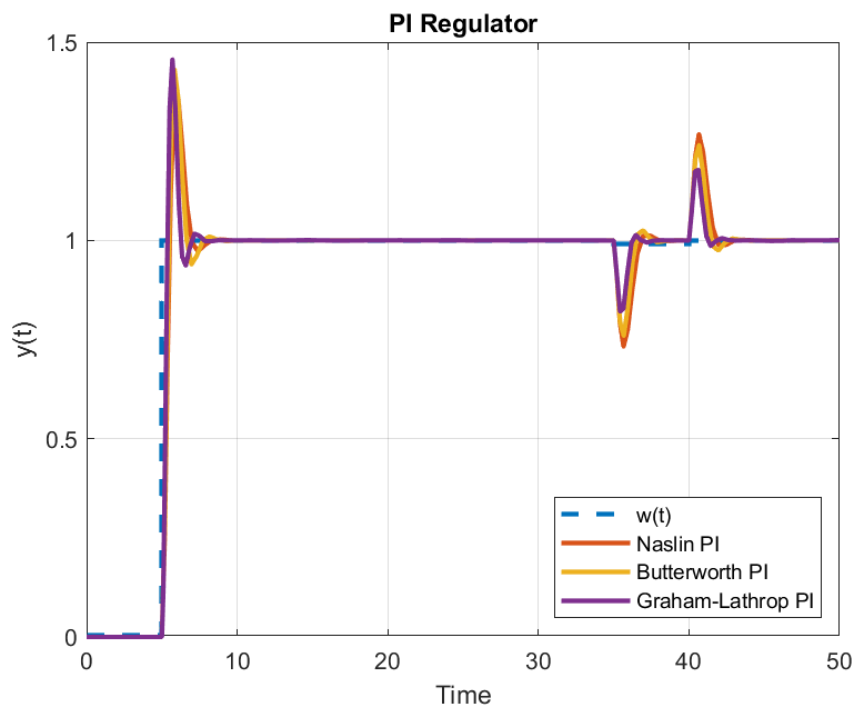
```

2.6 Vystupy zo simulacie

2.6.1 Výstup $y(t)$ – porovnanie: piecka vs motor (spojitý PI regulátor)

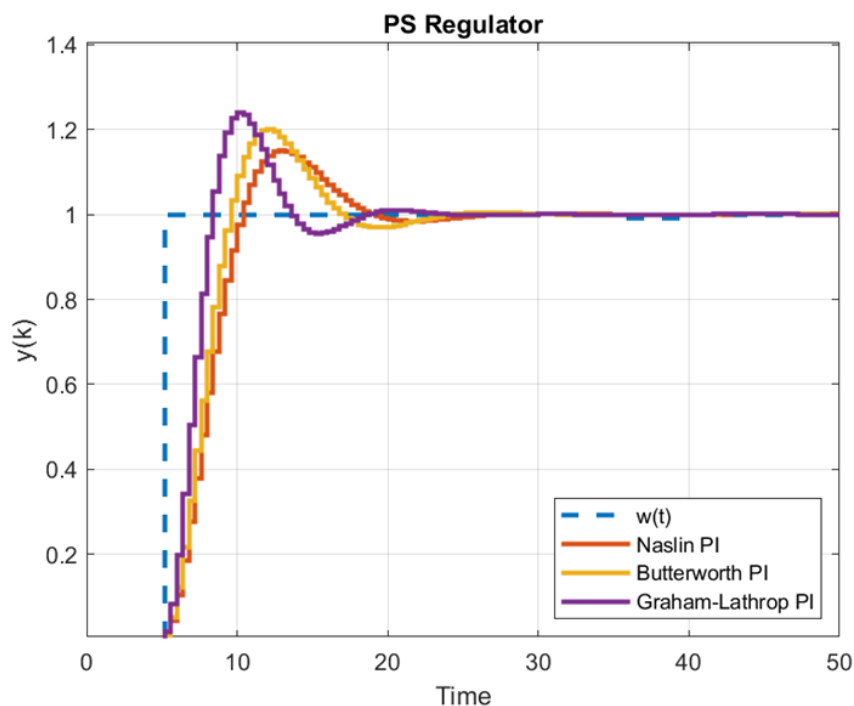


Obr. 6: Odozva výstupu $y(t)$ pre systém piecka pri rôznych PI regulátoroch. Prekmit a ustálenie závisia od zvolenej metódy návrhu.

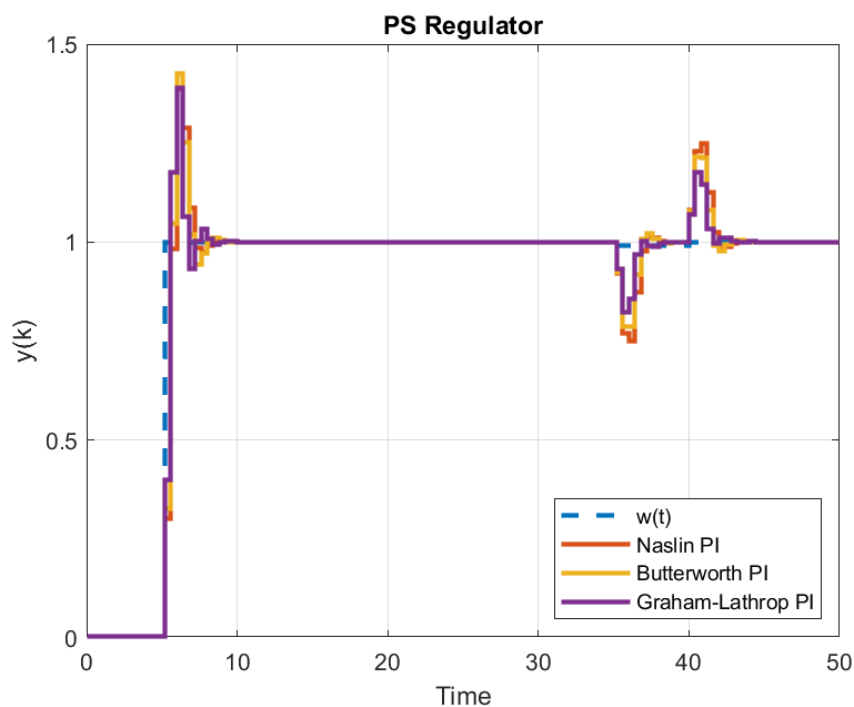


Obr. 7: Odozva výstupu $y(t)$ pre systém motor pri rovnakých PI regulátoroch. Dynamika je podobná ako pri piecke, s rozdielom v čase odozvy.

2.6.2 Výstup $y(k)$ – porovnanie: piecka vs motor (diskrétny PSD regulátor)



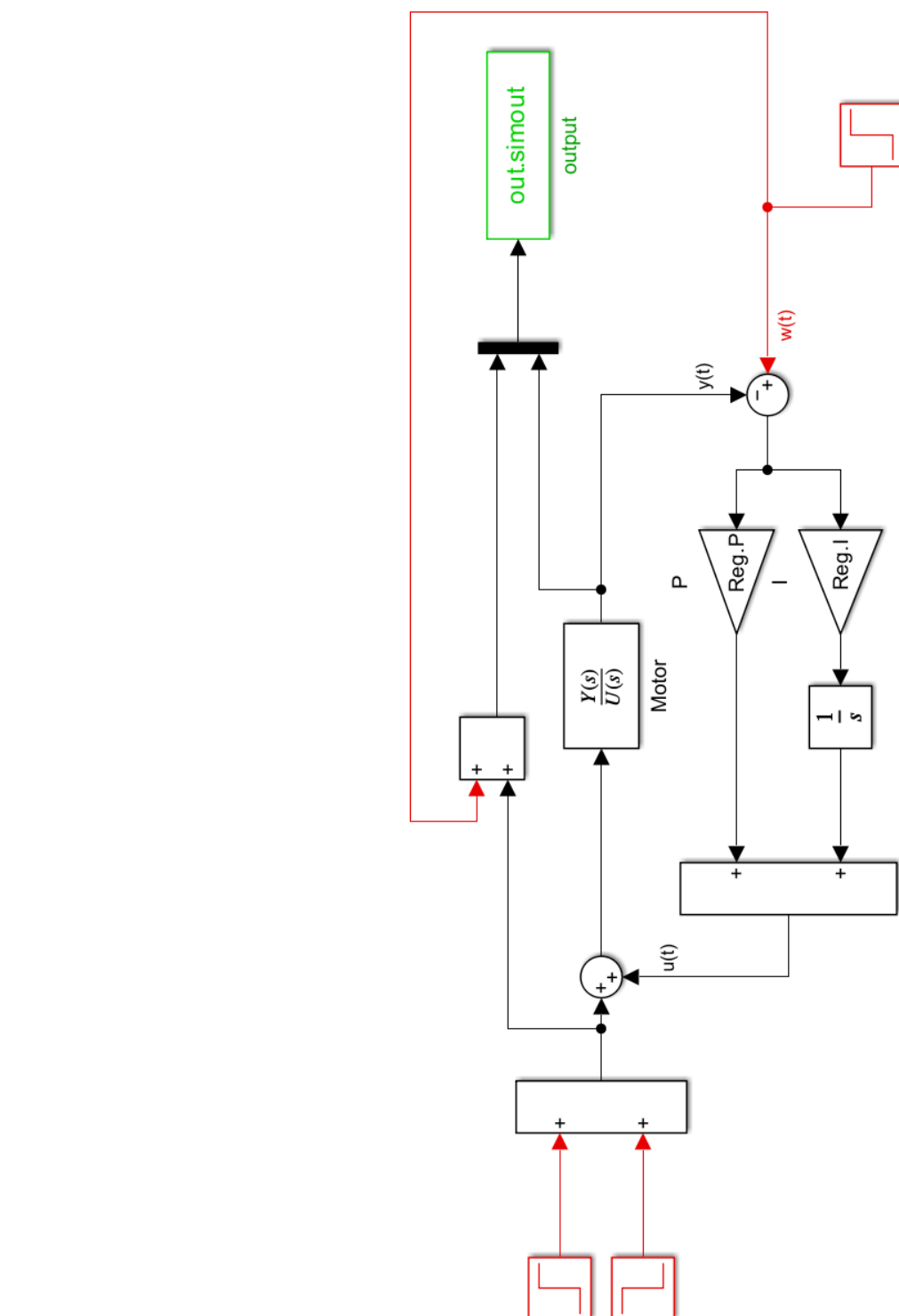
Obr. 8: Diskrétna odozva $y(k)$ systému piecka pri použití PSD regulátorov odvodených z PI. Viditeľné sú mierne odchýlky voči spojitému prípadu.



Obr. 9: Výstup $y(k)$ pre systém motor s rovnakými PSD regulátormi. Výsledky korešpondujú s teoretickou analýzou diskretizácie.

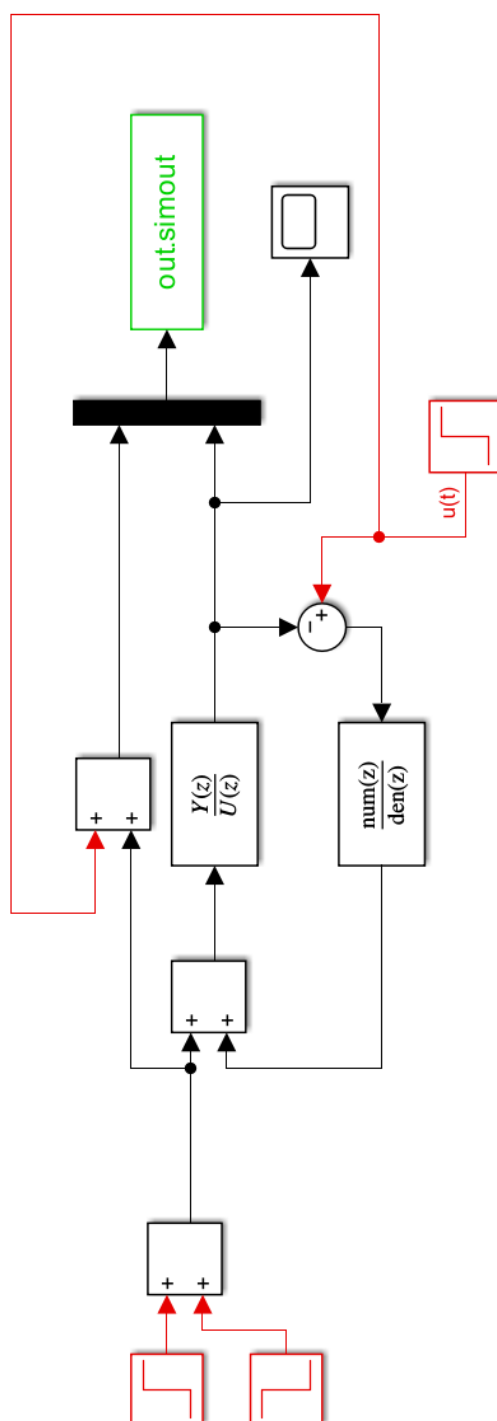
2.7 Modely regulačných štruktúr

2.7.1 Simulink model – PI regulátor (spojitý čas)



Obr. 10: Simulink model – uzavretá riadiaca štruktúra s PI regulátorom pre motor (reg.slx)

2.7.2 Simulink model – PSD regulátor (diskrétny čas)



Obr. 11: Simulink model – diskretna riadiaca štruktúra s PSD regulátorom pre motor (dreg.slx)

3 Prípadová štúdia 3 – Piecka, Motor, Výmenník

3.1 Znenie

Cieľom tejto úlohy je navrhnúť a overiť diskretný regulátor typu **dead-beat** (DB) pre tri diskretné lineárne dynamické systémy (LDS):

- systém piecky 2. rádu (druhého rádu),
- systém jednosmerného motora 2. rádu,
- systém výmenníka 3. rádu (podľa prednášky 4, slide 16).

Úlohy sa skladajú z nasledovných bodov:

- Návrh dead-beat regulátora:** Určiť koeficienty diskretného regulátora DB na konečný počet krokov bez obmedzenia a s obmedzením na riadiaci zásah $u(0)$. Implementovať ako MATLAB funkciu.
- Analytické výpočty prechodových dejov:** Vypočítať prechodový dej regulovanej veličiny $y(k)$, akčnej veličiny $u(k)$ a regulačnej odchýlky $e(k)$ analyticky (rekurzívne) a overiť ich simuláciou.
- Výpočet ustálených hodnôt:** Určiť ustálené hodnoty $y(\infty)$, $u(\infty)$, $e(\infty)$ na základe simulácie a analytického výpočtu pomocou samostatnej MATLAB funkcie.
- Overenie regulačného obvodu v Simulinku:** V uzavretej regulačnej štruktúre URO s DB regulátorom overiť dosiahnutie požadovanej hodnoty za konečný počet krokov pre rôzne priebehy riadiacej veličiny $w(k)$ a poruchy $z(k)$. Vizualizovať prechodové deje v systéme $y(k)$, $u(k)$ a $e(k)$ pomocou blokového modelu v Simulinku (so/bez saturácie).

3.2 Analytické riešenie

3.3 Hlavné skripty pre jednotlivé systémy

main_motor.m – Motor s dead-beat regulátorom Skript vykonáva návrh regulátora pre model motora, vykoná simulačný prechod so a bez saturácie, a zobrazuje priebehy veličí.

```
1 clear; clc;
2
3 [a, b] = motor(); % z 'systems/motor.m'
4 dtf.num = [0, b]; % z-1 a z-2
5 dtf.den = [1, a]; % z0, z-1, z-2
6
7 Reg = DBReg(dtf);
8 disp('Dead-beat bez saturacie:');
9 disp('Q(z):');
10 disp(Reg.num);
11 disp('P(z):');
12 disp(Reg.den);
13
14 u0 = 0.2;
15 RegSat = DBRegRest(dtf, u0);
16 disp(['Dead-beat so saturaciou (u(0)=', num2str(u0), '):']);
17 disp('Q(z):');
18 disp(RegSat.num);
19 disp('P(z):');
20 disp(RegSat.den);
21
22 N = 20;
23
24 w = ones(1, N);
25
26 [y1, u1, e1] = simulate_response(a, b, Reg, w, N);
27
28 [y2, u2, e2] = simulate_response(a, b, RegSat, w, N);
29
30 figure;
31 subplot(3,1,1); plot(0:N-1, y1, 'b', 0:N-1, y2, 'r--'); title('y(
    k)'); legend('bez obmedzenia', 's obmedzenim');
32 subplot(3,1,2); plot(0:N-1, u1, 'b', 0:N-1, u2, 'r--'); title('u(
    k)');
33 subplot(3,1,3); plot(0:N-1, e1, 'b', 0:N-1, e2, 'r--'); title('e(
    k)');
34 xlabel('k');
35
36 [y_inf1, u_inf1, e_inf1] = compute_steady_state(y1, u1, e1);
37 [y_inf2, u_inf2, e_inf2] = compute_steady_state(y2, u2, e2);
38
39 fprintf('\n--- USTALENE HODNOTY ---\n');
40
41 tspan = 2;
42 desr = [1 1.1];
43 desr_val = 0.01;
```

```

44
45 % Load both simulations
46 Data      = sim("regg.slx");      % without saturation
47 DataSat    = sim("reg_sat.slx"); % with saturation
48
49 % Time vector
50 nas_t = Data.tout;
51
52 % Extract signals
53 w_t      = Data.simout.Data(:, 1);
54 y_t      = Data.simout.Data(:, 2);
55 u_t      = Data.simout.Data(:, 3);
56 e_t      = Data.simout.Data(:, 4);
57
58 y_t_sat = DataSat.simout.Data(:, 2);
59 u_t_sat = DataSat.simout.Data(:, 3);
60 e_t_sat = DataSat.simout.Data(:, 4);
61
62 % Plot y(t)
63 figure;
64 stairs(nas_t, y_t, 'LineWidth', 2, 'Color', "r");
65 hold on;
66 stairs(nas_t, y_t_sat, 'LineWidth', 2, 'Color', "g");
67 stairs(nas_t, w_t, 'LineWidth', 2, 'Color', "b");
68 legend("y(t) - bez obmedzenia", "y(t) - s obmedzenim", "w(t)");
69 title("Motor y(t) - Dead-Beat Regulator");
70 xlabel("Cas [s]");
71 ylabel("y(t)");
72 grid on;
73
74 % Plot u(t)
75 figure;
76 stairs(nas_t, u_t, 'LineWidth', 2, 'Color', "r");
77 hold on;
78 stairs(nas_t, u_t_sat, 'LineWidth', 2, 'Color', "g");
79 stairs(nas_t, w_t, 'LineWidth', 2, 'Color', "b");
80 legend("u(t) - bez obmedzenia", "u(t) - s obmedzenim", "w(t)");
81 title("Motor Riadiaca velicina u(t)");
82 xlabel("Cas [s]");
83 ylabel("u(t)");
84 grid on;
85
86 % Plot e(t)
87 figure;
88 stairs(nas_t, e_t, 'LineWidth', 2, 'Color', "r");
89 hold on;
90 stairs(nas_t, e_t_sat, 'LineWidth', 2, 'Color', "g");
91 stairs(nas_t, w_t, 'LineWidth', 2, 'Color', "b");
92 legend("e(t) - bez obmedzenia", "e(t) - s obmedzenim", "w(t)");
93 title("Motor Reg odchylka e(t)");
94 xlabel("Cas [s]");

```

```

95 ylabel("e(t)");
96 grid on;

```

main_pec.m – Piecka s dead-beat regulátorom Analogický postup pre piecku:

```

1  clear; clc;
2
3  [a, b] = pec();
4  dtf.num = [0, b];
5  dtf.den = [1, a];
6
7  % --- DEAD-BEAT BEZ OBMEDZENIA ---
8  Reg = DBReg(dtf);
9  disp('Dead-beat bez saturacie:');
10 disp('Q(z):');
11 disp(Reg.num);
12 disp('P(z):');
13 disp(Reg.den);
14
15 % --- DEAD-BEAT S OBMEDZENIM NA u(0) ---
16 u0 = 0.2;
17 RegSat = DBRegRest(dtf, u0);
18 disp(['Dead-beat so saturaciou u(0) = ', num2str(u0), ']):');
19 disp('Q(z):');
20 disp(RegSat.num);
21 disp('P(z):');
22 disp(RegSat.den);
23
24 N = 20;
25
26 w = ones(1, N);
27
28 [y1, u1, e1] = simulate_response(a, b, Reg, w, N);
29
30 [y2, u2, e2] = simulate_response(a, b, RegSat, w, N);
31
32 figure;
33 subplot(3,1,1); plot(0:N-1, y1, 'b', 0:N-1, y2, 'r--'); title('y(
    k)'); legend('bez obmedzenia', 's obmedzenim');
34 subplot(3,1,2); plot(0:N-1, u1, 'b', 0:N-1, u2, 'r--'); title('u(
    k)');
35 subplot(3,1,3); plot(0:N-1, e1, 'b', 0:N-1, e2, 'r--'); title('e(
    k)');
36 xlabel('k');
37
38 [y_inf1, u_inf1, e_inf1] = compute_steady_state(y1, u1, e1);
39 [y_inf2, u_inf2, e_inf2] = compute_steady_state(y2, u2, e2);
40
41 fprintf('\n--- USTALENE HODNOTY ---\n');
42
43

```

```

44 tspan = 2;
45 desr = [1 1.1];
46 desr_val = 0.01;
47
48 % Load both simulations
49 Data      = sim("regg.slx");      % without saturation
50 DataSat   = sim("reg_sat.slx"); % with saturation
51
52 % Time vector
53 nas_t = Data.tout;
54
55 % Extract signals
56 w_t     = Data.simout.Data(:, 1);
57 y_t     = Data.simout.Data(:, 2);
58 u_t     = Data.simout.Data(:, 3);
59 e_t     = Data.simout.Data(:, 4);
60
61 y_t_sat = DataSat.simout.Data(:, 2);
62 u_t_sat = DataSat.simout.Data(:, 3);
63 e_t_sat = DataSat.simout.Data(:, 4);
64
65 % Plot y(t)
66 figure;
67 stairs(nas_t, y_t, 'LineWidth', 2, 'Color', "r");
68 hold on;
69 stairs(nas_t, y_t_sat, 'LineWidth', 2, 'Color', "g");
70 stairs(nas_t, w_t, 'LineWidth', 2, 'Color', "b");
71 legend("y(t) - bez obmedzenia", "y(t) - s obmedzenim", "w(t)");
72 title("Piecka y(t) - Dead-Beat Regulator");
73 xlabel("Cas [s]");
74 ylabel("y(t)");
75 grid on;
76
77 % Plot u(t)
78 figure;
79 stairs(nas_t, u_t, 'LineWidth', 2, 'Color', "r");
80 hold on;
81 stairs(nas_t, u_t_sat, 'LineWidth', 2, 'Color', "g");
82 stairs(nas_t, w_t, 'LineWidth', 2, 'Color', "b");
83 legend("u(t) - bez obmedzenia", "u(t) - s obmedzenim", "w(t)");
84 title("Piecka Riadiaca velicina u(t)");
85 xlabel("Cas [s]");
86 ylabel("u(t)");
87 grid on;
88
89 % Plot e(t)
90 figure;
91 stairs(nas_t, e_t, 'LineWidth', 2, 'Color', "r");
92 hold on;
93 stairs(nas_t, e_t_sat, 'LineWidth', 2, 'Color', "g");
94 stairs(nas_t, w_t, 'LineWidth', 2, 'Color', "b");

```

```

95 legend("e(t) - bez obmedzenia", "e(t) - s obmedzenim", "w(t)");
96 title("Piecka Regulacne odchylka e(t)");
97 xlabel("Cas [s]");
98 ylabel("e(t)");
99 grid on;

```

main_vymenik.m – Vymeník (3. rád) Model vymeníka s vyšším rádom, ale rovnakým postupom návrhu regulátora:

```

1 clear; clc;
2
3 [a, b] = vymenik();
4 dtf.num = [0, b];
5 dtf.den = [1, a];
6
7 % --- DEAD-BEAT BEZ OBMEDZENIA ---
8 Reg = DBReg(dtf);
9 disp('Dead-beat bez saturacie:');
10 disp('Q(z):');
11 disp(Reg.num);
12 disp('P(z):');
13 disp(Reg.den);
14
15 % --- DEAD-BEAT S OBMEDZENIM NA u(0) ---
16 u0 = 0.2;
17 RegSat = DBRegRest(dtf, u0);
18 disp(['Dead-beat so saturaciou (u(0) = ', num2str(u0), '):']);
19 disp('Q(z):');
20 disp(RegSat.num);
21 disp('P(z):');
22 disp(RegSat.den);
23
24 N = 20;
25
26 w = ones(1, N);
27
28 [y1, u1, e1] = simulate_response(a, b, Reg, w, N);
29
30 [y2, u2, e2] = simulate_response(a, b, RegSat, w, N);
31
32 figure;
33 subplot(3,1,1); plot(0:N-1, y1, 'b', 0:N-1, y2, 'r--'); title('y(k)'); legend('bez obmedzenia', 's obmedzenim');
34 subplot(3,1,2); plot(0:N-1, u1, 'b', 0:N-1, u2, 'r--'); title('u(k)');
35 subplot(3,1,3); plot(0:N-1, e1, 'b', 0:N-1, e2, 'r--'); title('e(k)');
36 xlabel('k');
37
38 [y_inf1, u_inf1, e_inf1] = compute_steady_state(y1, u1, e1);
39 [y_inf2, u_inf2, e_inf2] = compute_steady_state(y2, u2, e2);

```

```

40
41 fprintf('\n---_USTALENE_HODNOTY_---\n');
42
43 tspan = 2;
44 desr = [1 1.1];
45 desr_val = 0.01;
46
47 % Load both simulations
48 Data      = sim("regg.slx");      % without saturation
49 DataSat   = sim("reg_sat.slx"); % with saturation
50
51 % Time vector
52 nas_t = Data.tout;
53
54 % Extract signals
55 w_t     = Data.simout.Data(:, 1);
56 y_t     = Data.simout.Data(:, 2);
57 u_t     = Data.simout.Data(:, 3);
58 e_t     = Data.simout.Data(:, 4);
59
60 y_t_sat = DataSat.simout.Data(:, 2);
61 u_t_sat = DataSat.simout.Data(:, 3);
62 e_t_sat = DataSat.simout.Data(:, 4);
63
64 % Plot y(t)
65 figure;
66 stairs(nas_t, y_t, 'LineWidth', 2, 'Color', "r");
67 hold on;
68 stairs(nas_t, y_t_sat, 'LineWidth', 2, 'Color', "g");
69 stairs(nas_t, w_t, 'LineWidth', 2, 'Color', "b");
70 legend("y(t) - bez obmedzenia", "y(t) - s obmedzenim", "w(t)");
71 title("Vymenik y(t) - Dead-Beat Regulator");
72 xlabel("Cas [s]");
73 ylabel("y(t)");
74 grid on;
75
76 % Plot u(t)
77 figure;
78 stairs(nas_t, u_t, 'LineWidth', 2, 'Color', "r");
79 hold on;
80 stairs(nas_t, u_t_sat, 'LineWidth', 2, 'Color', "g");
81 stairs(nas_t, w_t, 'LineWidth', 2, 'Color', "b");
82 legend("u(t) - bez obmedzenia", "u(t) - s obmedzenim", "w(t)");
83 title("Vymenik Riadiaca velicina u(t)");
84 xlabel("Cas [s]");
85 ylabel("u(t)");
86 grid on;
87
88 % Plot e(t)
89 figure;
90 stairs(nas_t, e_t, 'LineWidth', 2, 'Color', "r");

```

```

91 hold on;
92 stairs(nas_t, e_t_sat, 'LineWidth', 2, 'Color', "g");
93 stairs(nas_t, w_t, 'LineWidth', 2, 'Color', "b");
94 legend("e(t) - bez obmedzenia", "e(t) - s obmedzenim", "w(t)");
95 title("Vymenik Regulacna odchylka e(t)");
96 xlabel("Cas [s]");
97 ylabel("e(t)");
98 grid on;

```

3.4 Použité MATLAB funkcie

Táto sekcia obsahuje všetky pomocné funkcie implementované v prostredí MATLAB, ktoré sú využité pri simulácii a výpočtoch v úlohách Prípadovej štúdie 3 (systémy piecka, motor, výmenník). Každá funkcia je doplnená o stručný popis jej účelu a vstupno-výstupných parametrov.

Funkcia motor.m

```

1 function [a, b] = motor()
2 % Diskretna prenosova funkcia jednosmerneho motora
3 a = [-1.58, 0.58];
4 b = [1.79, 1.5];
5 end

```

Listing 20: Definícia diskretizovaného modelu motora

Funkcia pec.m

```

1 function [a, b] = pec()
2 % Diskretna prenosova funkcia pre system piecky
3 a = [-1.2, 0.3];
4 b = [0.8, 0.1];
5 end

```

Listing 21: Definícia systému pece 2. rádu

Funkcia vymenik.m

```

1 function [a, b] = vymenik()
2 Ts = 10;
3 Gs = tf(1, conv([3 1], conv([3 1], [3 1])));
4 Gz = c2d(Gs, Ts, 'tustin');
5 [num, den] = tfdata(Gz, 'v');
6 b = num(2:end);
7 a = den(2:end);
8 end

```

Listing 22: Diskretizácia systému výmenníka 3. rádu

Funkcia DBReg.m

```
1 function Reg = DBReg(dtf)
2 b = dtf.num;
3 a = dtf.den;
4 B = b(2:end);
5 A = a(2:end);
6 M = length(B);
7 q0 = 1 / sum(B);
8 Q = zeros(1, M+1); Q(1) = q0;
9 for i = 1:M, Q(i+1) = A(i) * q0; end
10 P = -B * q0; P = [1 P];
11 Reg.num = Q; Reg.den = P;
12 end
```

Listing 23: Výpočet dead-beat regulátora bez saturácie

Funkcia DBRegRest.m

```
1 function Reg = DBRegRest(dtf, u0)
2 b = dtf.num; a = dtf.den;
3 A = a(2:end); B = b(2:end);
4 M = length(B); Bsum = sum(B);
5 q0 = u0; Q = zeros(1, M+2); Q(1) = q0;
6 for i = 1:M
7     if i == 1, Q(i+1) = (A(i) - 1) * q0 + 1 / Bsum;
8     else, Q(i+1) = q0 * (A(i) - A(i-1)) + A(i-1) / Bsum;
9     end
10 end
11 Q(M+2) = A(M) * (-q0 + 1 / Bsum);
12 P = zeros(1, M+1);
13 for i = 1:M
14     if i == 1, P(i) = B(i) * q0;
15     else, P(i) = q0 * (B(i) - B(i-1)) + B(i-1) / Bsum;
16     end
17 end
18 P(M+1) = -B(M) * (q0 - 1 / Bsum); P = [1 -P];
19 Reg.num = Q; Reg.den = P;
20 end
```

Listing 24: Dead-beat regulátor s obmedzením na $u(0)$

Funkcia simulate_response.m

```
1 function [y, u, e] = simulate_response(a, b, Reg, w, N)
2 na = length(a); nb = length(b);
3 nq = length(Reg.num); np = length(Reg.den) - 1;
4 y = zeros(1, N); u = zeros(1, N); e = zeros(1, N);
5 for k = 1:N
6     e(k) = w(k) - y(k);
```

```

7      uk = 0;
8      for i = 1:min(np, k-1), uk = uk + (-Reg.den(i+1)) * u(k-i);
          end
9      for i = 1:min(nq, k), uk = uk + Reg.num(i) * e(k - i + 1);
          end
10     u(k) = uk;
11     yk = 0;
12     for i = 1:min(na, k-1), yk = yk - a(i) * y(k-i); end
13     for i = 1:min(nb, k), yk = yk + b(i) * u(k - i + 1); end
14     y(k) = yk;
15 end
16 end

```

Listing 25: Simulácia systému so zadaným regulátorom

Funkcia compute_steady_state.m

```

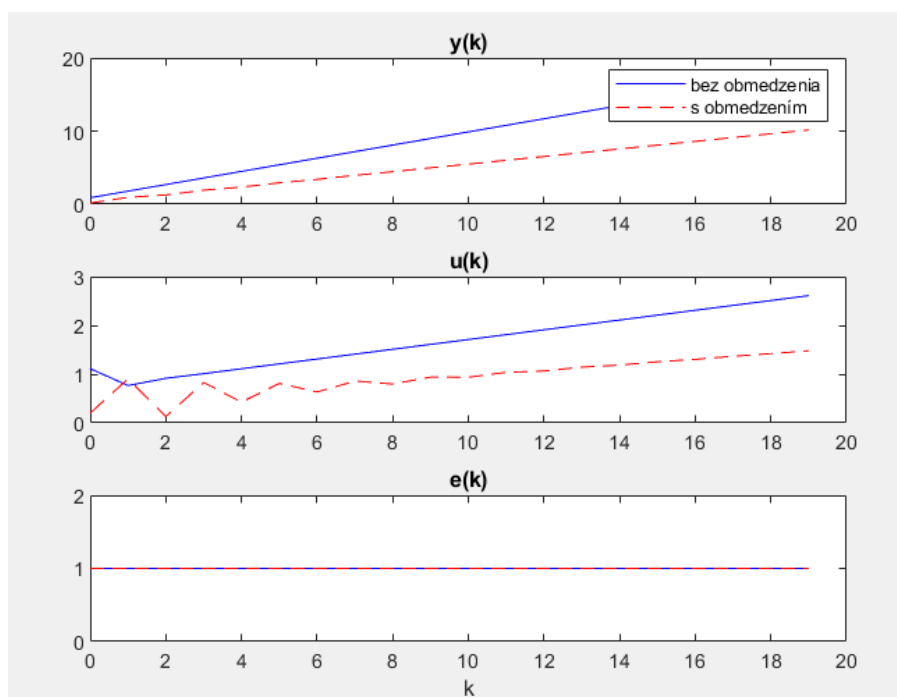
1 function [y_inf, u_inf, e_inf] = compute_steady_state(y, u, e)
2 tail = round(length(y) * 0.2);
3 y_inf = mean(y(end-tail+1:end));
4 u_inf = mean(u(end-tail+1:end));
5 e_inf = mean(e(end-tail+1:end));
6 end

```

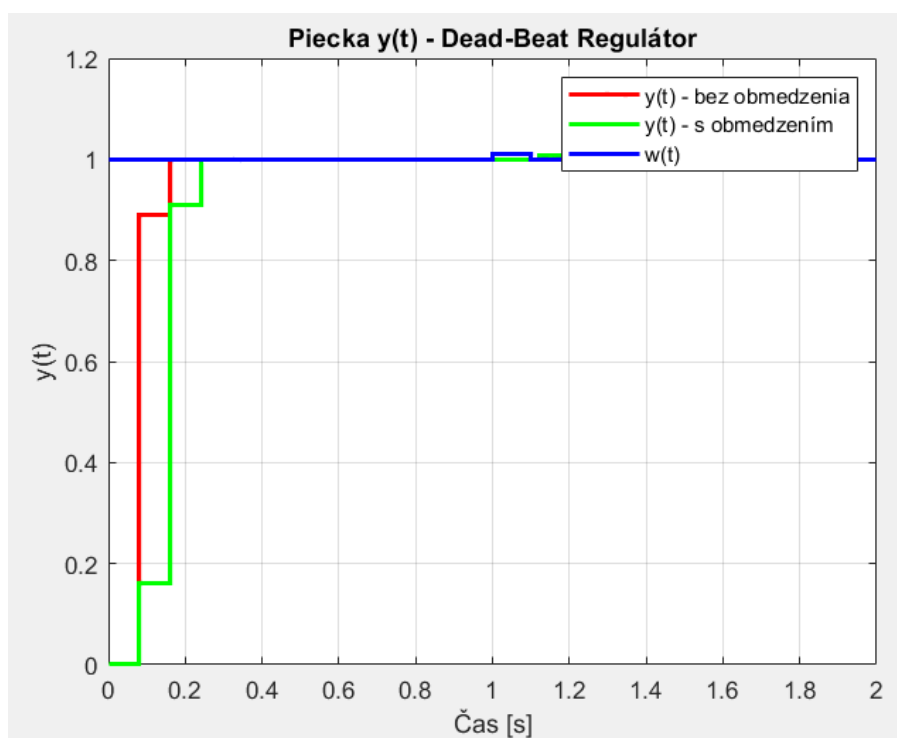
Listing 26: Výpočet ustálených hodnôt

3.5 Výstupy zo simulácie

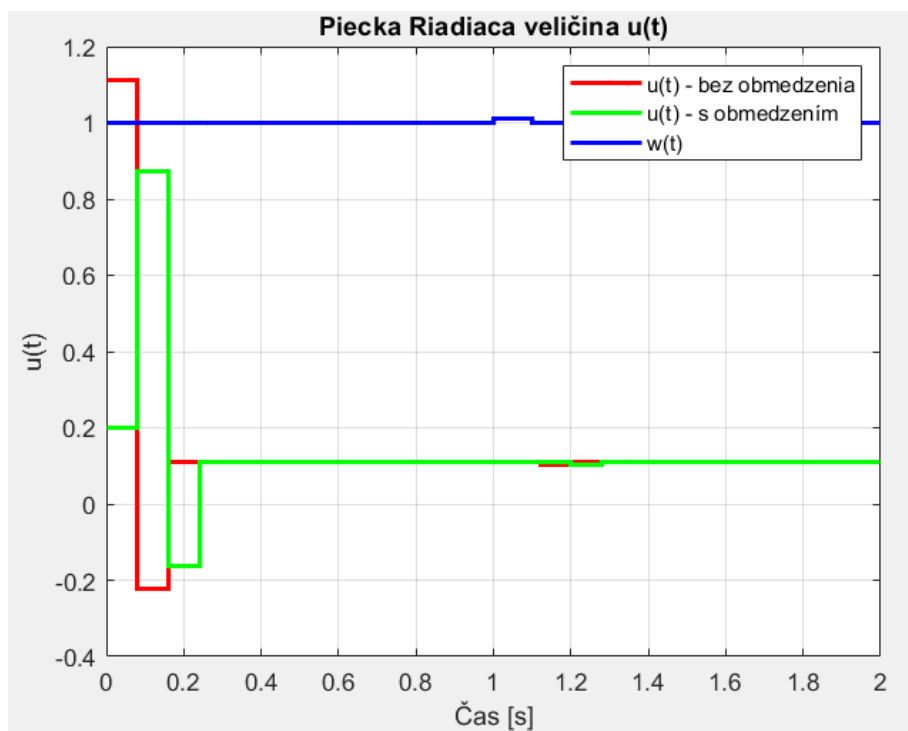
Piecka



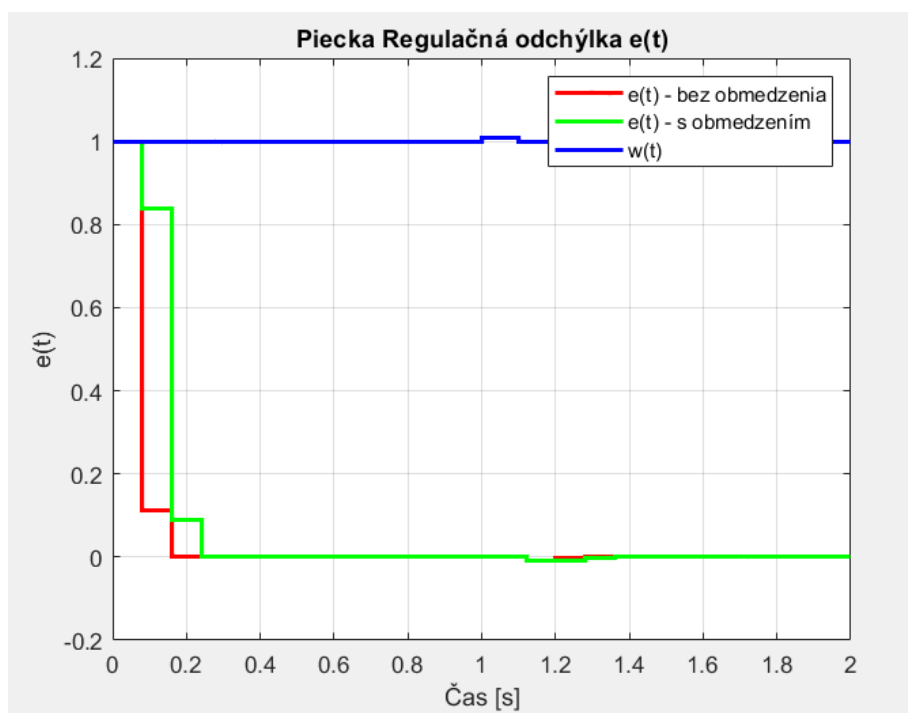
Obr. 12: Prechodový dej $y(k)$, $u(k)$, $e(k)$ pre systém piecky



Obr. 13: Výstup $y(t)$ – Dead-Beat regulátor pre piecku

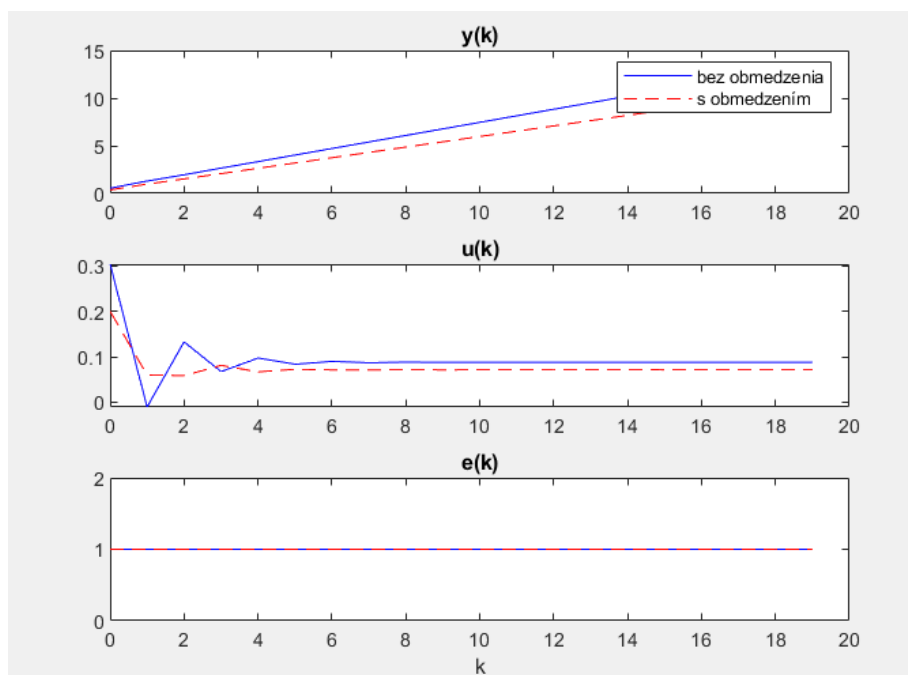


Obr. 14: Riadiaca veličina $u(t)$ pre systém piecky

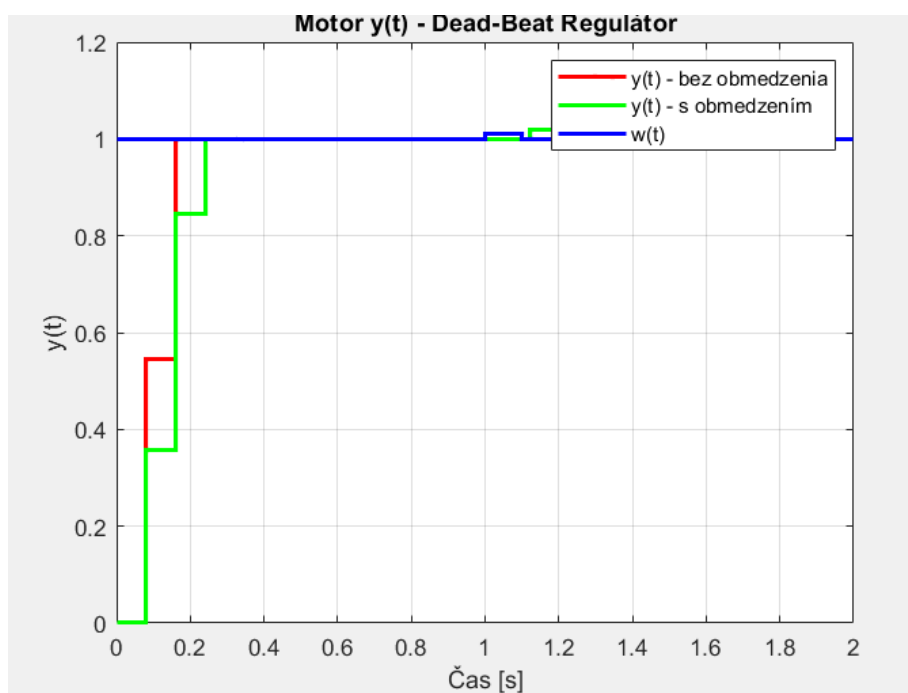


Obr. 15: Regulačná odchýlka $e(t)$ pre piecku

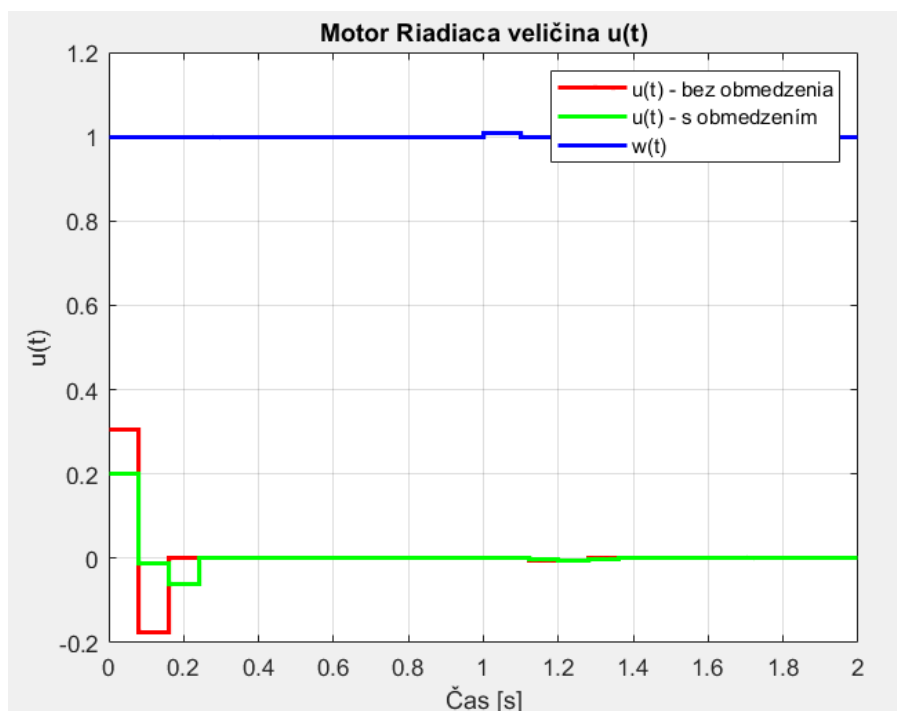
Motor



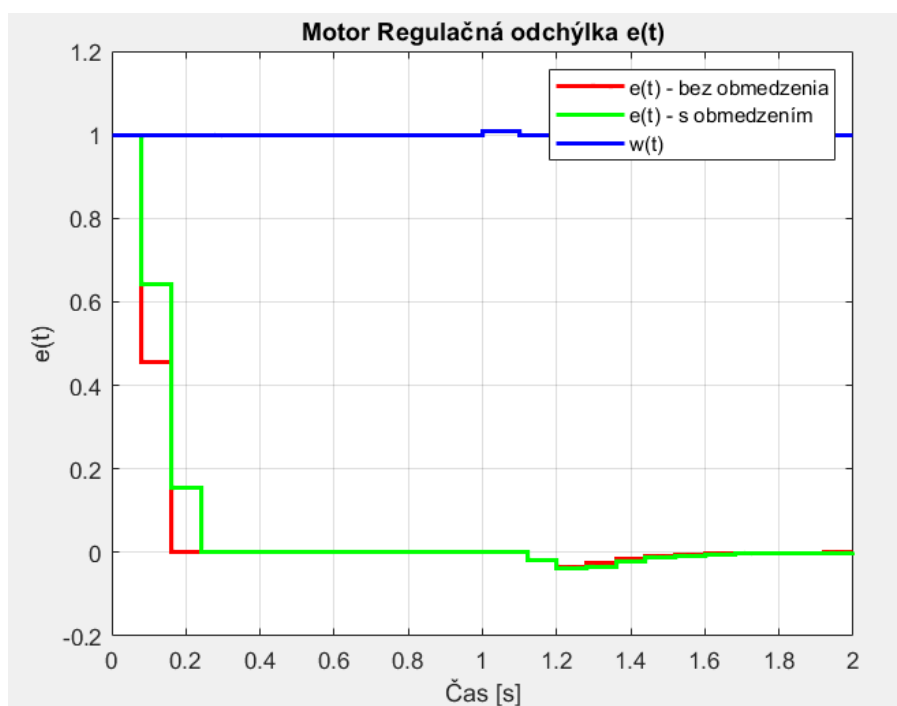
Obr. 16: Prechodový dej $y(k)$, $u(k)$, $e(k)$ pre motor



Obr. 17: Výstup $y(t)$ – Dead-Beat regulátor pre motor

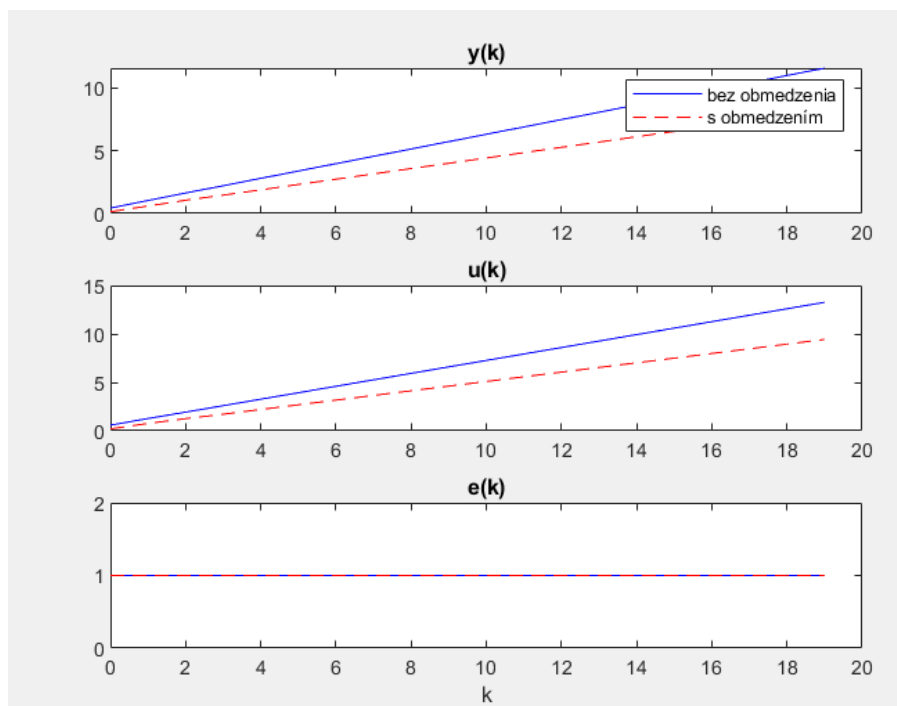


Obr. 18: Riadiaca veličina $u(t)$ pre motor

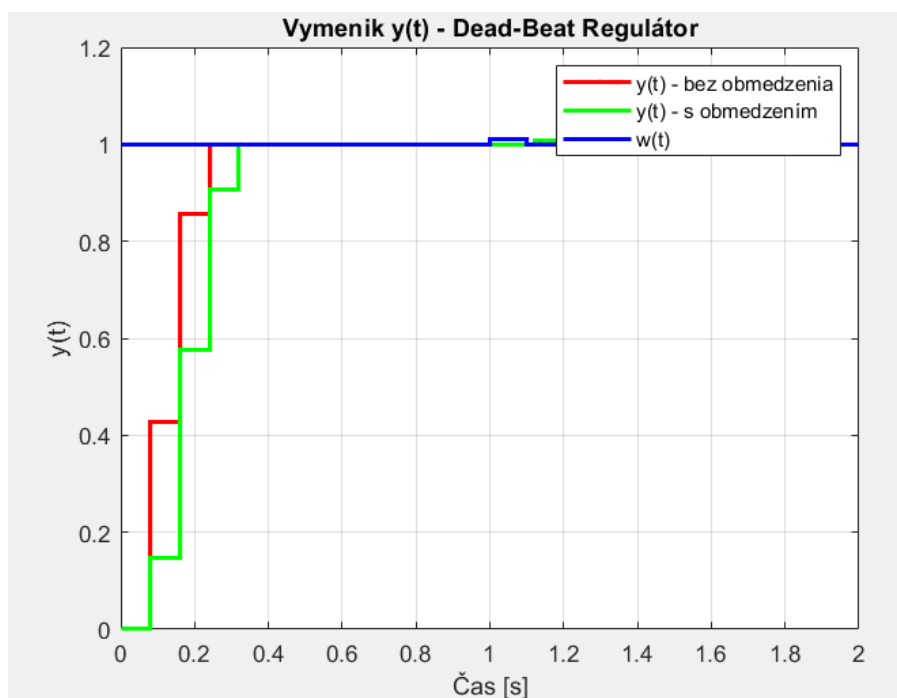


Obr. 19: Regulačná odchýlka $e(t)$ pre motor

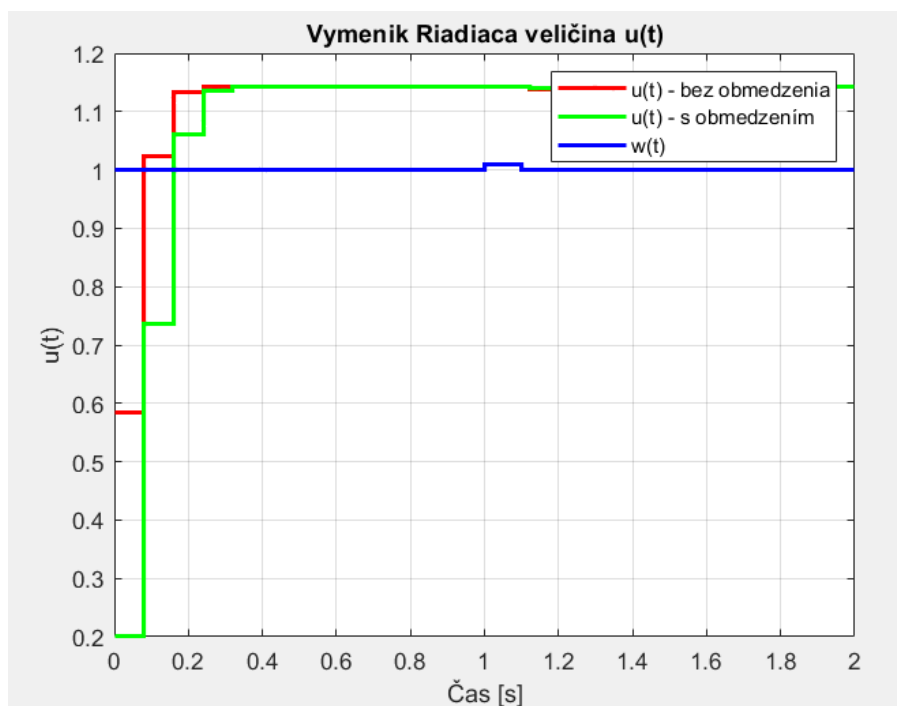
Výmenník



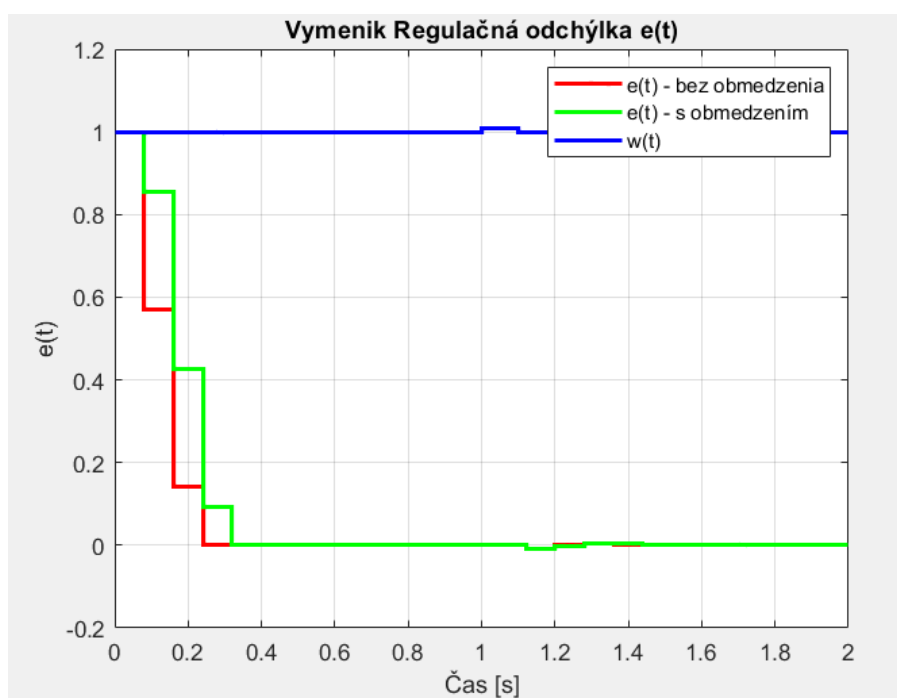
Obr. 20: Prechodový dej $y(k)$, $u(k)$, $e(k)$ pre výmenník



Obr. 21: Výstup $y(t)$ – Dead-Beat regulátor pre výmenník

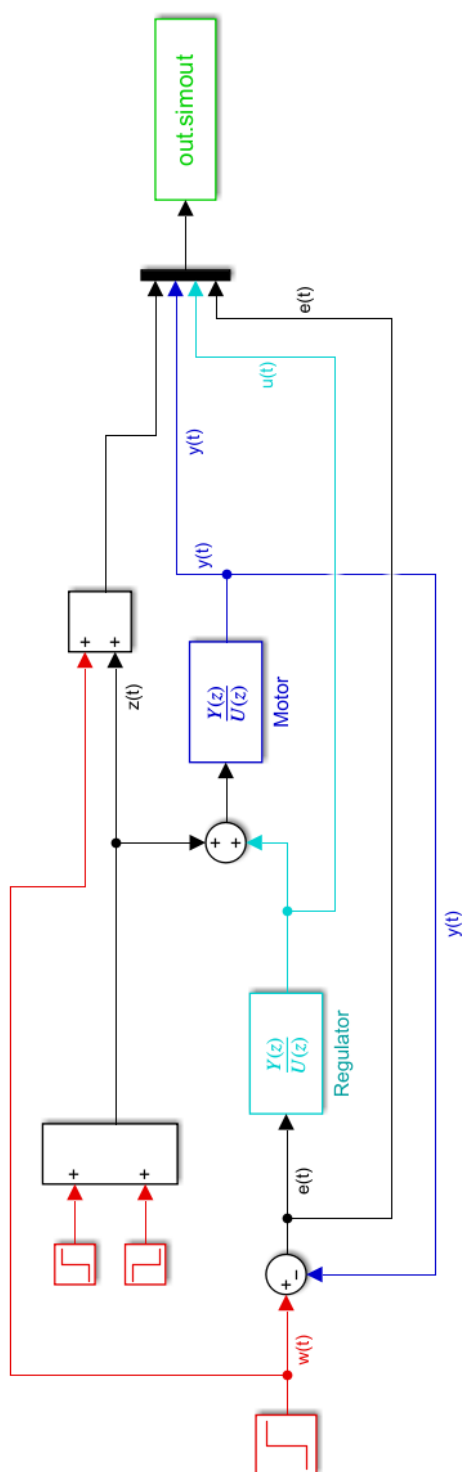


Obr. 22: Riadiaca veličina $u(t)$ pre výmenník

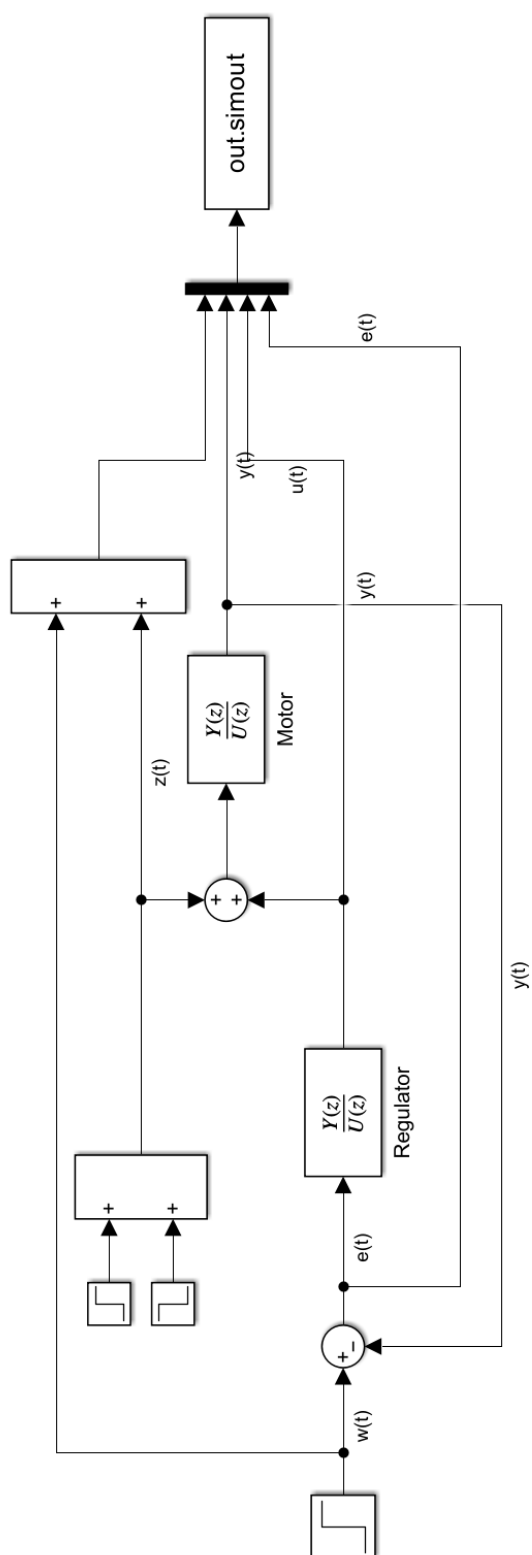


Obr. 23: Regulačná odchýlka $e(t)$ pre výmenník

3.6 Modely regulačných štruktúr v Simulinku



Obr. 24: Simulink model URO bez obmedzenia (dead-beat regulátor)



Obr. 25: Simulink model URO s obmedzením na $u(0)$

4 Prípadová štúdia 4 – Jednosmerný motor

4.1 Znenie

V tejto prípadovej štúdii je úlohou navrhnuť diskretný regulátor pre jednosmerný motor s využitím metódy predpisania pólov (Pole Placement – PP). Riešenie sa zameriava na riadenie uhlovej rýchlosti motora v uzavretej regulačnej slučke.

Zadanie pozostáva z nasledovných úloh:

- **a)** Pre systém jednosmerného motora určiť koeficienty diskretného regulátora na základe metódy predpisania pólov. Návrh má byť realizovaný výpočtom a podporený implementáciou príslušnej funkcie v prostredí MATLAB.
- **b)** V uzavretej regulačnej štruktúre s polynomiálnym regulátorom overiť riadenie pre zvolený referenčný signál $w(k)$ a poruchu $z(k)$, ak je prítomná. Výsledky prechodových dejov systému (t. j. priebehy veličín $y(k)$, $u(k)$ a $e(k)$) je potrebné vizualizovať na grafoch a porovnať ich s požadovaným správaním.

Poznámka: V prípade jednoduchej verzie skúšky študent rieši iba návrh regulátora pomocou PP metódy pre riadenie uhlovej rýchlosti motora. Dokumentácia sa sústreďuje len na sumarizáciu výsledkov simulácie.

4.2 Analytické riešenie

4.3 Programový kód (Matlab/Simulink)

Pre návrh regulátora metódou predpisania pólov (Pole Placement – PP) bol vytvorený nasledujúci hlavný skript:

Hlavný skript main.m

```
1 % Prenosova funkcia motora
2 [a, b] = motor();
3 dtf.num = b;
4 dtf.den = [1, a];
5
6 % Predpisane poly uzavreteho obvodu
7 poles = [0.1, 0.2, 0.3, 0.4]; % alebo ine
8
9 % Vepocet regulatora
10 Reg = ppc(dtf, poles);
11
12 disp('Polynomialny regulator:');
13 disp('Q(z):'); disp(Reg.num);
14 disp('P(z):'); disp(Reg.den);
15
16 data = sim("reggg.slx");
17 nas_t = data.tout;
18 w_t = data.simout.Data(:, 1);
19 y_t = data.simout.Data(:, 2);
20 u_t = data.simout.Data(:, 3);
21 e_t = data.simout.Data(:, 4);
22
23 % Vystupna velicina y(t)
24 figure;
25 stairs(nas_t, y_t, 'LineWidth', 2, 'Color', "r");
26 hold on;
27 stairs(nas_t, w_t, 'LineWidth', 2, 'Color', "b");
28 legend("y(t)", "w(t)");
29 title("PP Regulator");
30 xlabel("Cas [s]");
31 ylabel("y(t)");
32 grid on;
33
34 % Riadiaca velicina u(t)
35 figure;
36 stairs(nas_t, u_t, 'LineWidth', 2, 'Color', "r");
37 hold on;
38 stairs(nas_t, w_t, 'LineWidth', 2, 'Color', "b");
39 legend("u(t)", "w(t)");
40 title("PP Regulator riadiaca velicina");
41 xlabel("Cas [s]");
42 ylabel("u(t)");
43 grid on;
44
```

```

45 % Regulacna odchylka e(t)
46 figure;
47 stairs(nas_t, e_t, 'LineWidth', 2, 'Color', "r");
48 hold on;
49 stairs(nas_t, w_t, 'LineWidth', 2, 'Color', "b");
50 legend("e(t)", "w(t)");
51 title("PP Regulator regulacna odchylka");
52 xlabel("Cas [s]");
53 ylabel("e(t)");
54 grid on;

```

Funkcia motor.m

```

1 function [a, b] = motor()
2 % MOTOR - Diskretna prenosova funkcia jednosmerneho motora
3 %  $G_p(z) = (1.79 z^{-1} + 1.5 z^{-2}) / (1 - 1.58 z^{-1} + 0.58 z^{-2})$ 
4
5     a = [-1.58, 0.58]; % menovatel A(z) bez  $z^0$ 
6     b = [1.79, 1.5];   % citatel B(z)
7 end

```

Funkcia ppc.m – návrh polynomiálneho regulátora

```

1 function Reg = ppc(dtf, poles)
2     syms q0 q1 q2 p1
3
4     B = dtf.num;
5     A = dtf.den;
6
7     Cd = poly(poles); % pozadovany charakteristicky polynom
8
9     a1 = A(2);
10    a2 = A(3);
11
12    b1 = B(1);
13    b2 = B(2);
14
15    c1 = Cd(2);
16    c2 = Cd(3);
17    c3 = Cd(4);
18    c4 = Cd(5);
19
20    % Rovnice pre vypocet koeficientov regulatora
21    eq1 = a1 - (1 + p1) + q0 * b1 == c1;
22    eq2 = a2 - a1 * (1 + p1) + p1 + q0 * b2 + q1 * b1 == c2;
23    eq3 = a1 * p1 - a2 * (1 + p1) + b2 * q1 + b1 * q2 == c3;
24    eq4 = p1 * a2 + b2 * q2 == c4;
25
26    % Riesenie sustavy

```

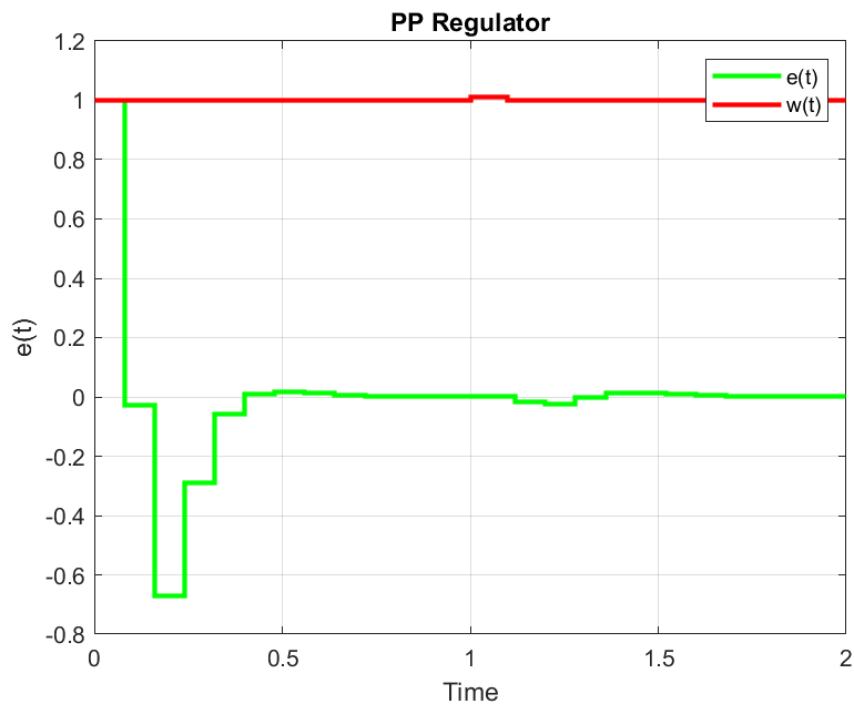
```

27     sol = solve([eq1, eq2, eq3, eq4], [q0, q1, q2, p1]);
28
29     q0 = double(sol.q0);
30     q1 = double(sol.q1);
31     q2 = double(sol.q2);
32     p1 = double(sol.p1);
33
34     Reg.num = [q0 q1 q2];
35     Reg.den = conv([1 -1], [1 -p1]);
36 end

```

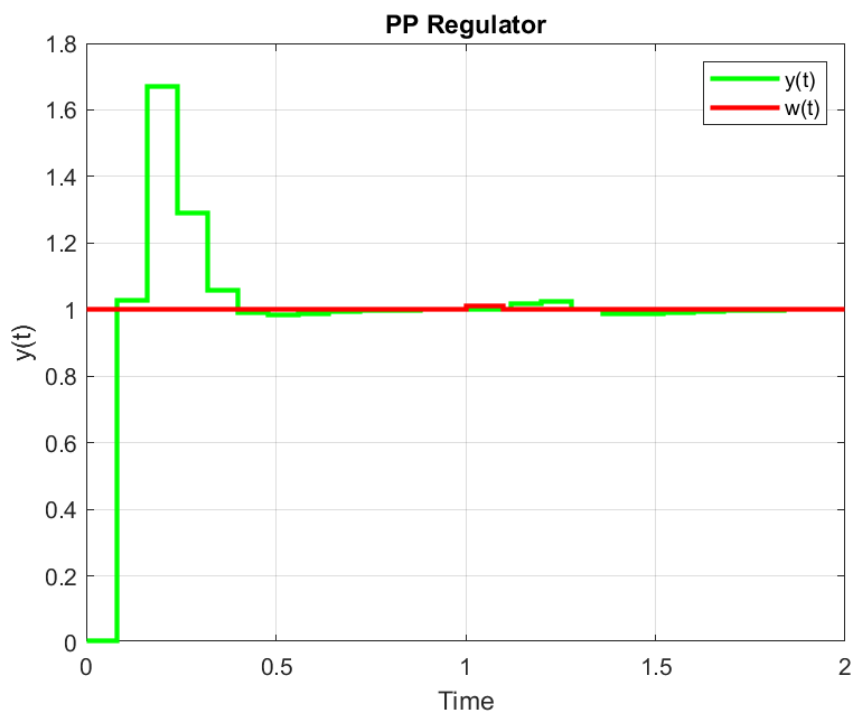
4.4 Výstupy zo simulácie

4.4.1 Odozva výstupnej veličiny $y(t)$



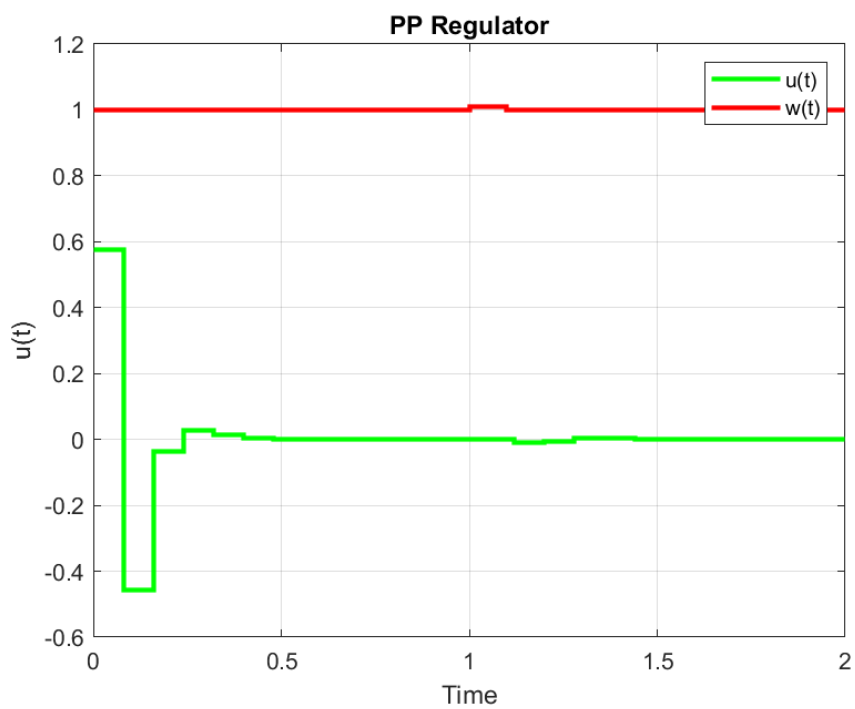
Obr. 26: Výstup $y(t)$ systému s polynomiálnym regulátorom (PP).

4.4.2 Riadiaca veličina $u(t)$



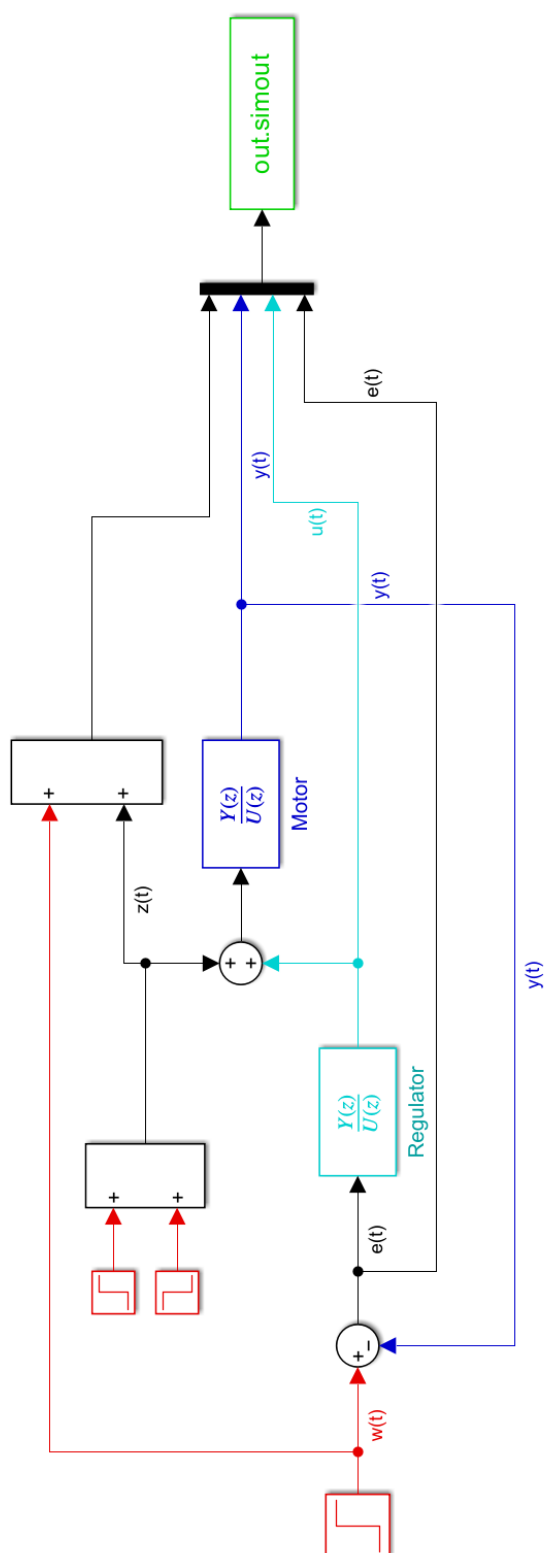
Obr. 27: Riadiaca veličina $u(t)$ pri použití PP regulátora.

4.4.3 Regulačná odchýlka $e(t)$



Obr. 28: Regulačná odchýlka $e(t)$ s PP regulátorom.

4.4.4 Simulink model – PP regulátor



Obr. 29: Simulink model uzavretej regulačnej štruktúry s polynomiálnym regulátorom.